



Escola Politècnica Superior
d'Enginyeria de Vilanova i la Geltrú

UNIVERSITAT POLITÈCNICA DE CATALUNYA

TREBALL FINAL DE GRAU

TÍTOL: Plataforma de Comerç Online per a la Venda de Cavalls

AUTORS: ONCINS SPEDO, FRANCESCO

DATA DE PRESENTACIÓ: OCTUBRE, 2025

COGNOMS: ONCINS SPEDO

NOM: FRANCESCO

TITULACIÓ: GRAU EN ENGINYERIA INFORMÀTICA

PLA: 2018

DIRECTOR: GARCIA ALMIÑANA, JORDI

COORDIRECTORA: LÓPEZ CUESTA, LIDIA

**DEPARTAMENT: DAC - DEPARTAMENT D'ARQUITECTURA DE
COMPUTADORS**

QUALIFICACIÓ DEL TFG

TRIBUNAL

PRESIDENT

SECRETÀRIA

VOCAL

SANCHEZ LOPEZ,

MARIN TORDERA,

MOTJER I CERVERO,

SERGIO

EVA

FRANCESC

DATA DE LECTURA: 27 D'OCTUBRE DE 2025

Aquest Projecte té en compte aspectes mediambientals: ☐ Sí ☐ No

RESUM

Aquest treball presenta el disseny i la implementació d'una plataforma de comerç online per a la venda de cavalls. L'objectiu és millorar la publicació, la descoberta i la gestió d'anuncis mitjançant filtres avançats, un mapa interactiu i una arquitectura clara basada en MVC (Model - Vista - Controlador). La solució separa clarament el frontend, backend i base de dades, amb una API REST documentada en OpenAPI/Swagger. El projecte ha seguit la metodologia Kanban per visualitzar fluxos, limitar WIP i ajustar prioritats en quatre fases.

Metodològicament, s'ha partit d'un estudi de mercat (Wallapop, Milanuncios, Coches.net, AutoScout24, ehorses, etc.) per identificar bones pràctiques i mancances (filtrat específic, geolocalització, agenda d'esdeveniments) i per delimitar la proposta de valor diferencial.

En la implementació, el backend exposa recursos d'usuaris, perfils, anuncis i disciplines amb rutes REST i /docs. El frontend consumeix l'API i renderitza llistats i fitxes amb filtres i mapa. El model de dades cobreix usuaris, sessions, anuncis, imatges, disciplines i revisions, amb restriccions semàntiques.

Els resultats acrediten el compliment dels requisits: registre i gestió d'usuaris, cicle de vida d'anuncis, validació per l'administrador, cerca amb filtres i mapa. Queden pendents "favorits", publicitat i gestió d'esdeveniments. Es conclou que l'arquitectura i el contracte d'API faciliten l'escalabilitat i mantenibilitat. Com a futur es pretén afegir esdeveniments amb calendari i geolocalització, patrocinis, favorits i notificacions, i millores d'accessibilitat com la integració de nous idiomes.

Paraules clau (màxim 10):

Web	Anuncis	Cavalls	MVC
JavaScript	MariaDB	Node.js	API
Geolocalització			

RESUMEN

Este trabajo presenta el diseño e implementación de una plataforma de comercio online para la venta de caballos. El objetivo es mejorar la publicación, la búsqueda y la gestión de anuncios mediante filtros avanzados, un mapa interactivo y una arquitectura clara basada en MVC (Modelo - Vista - Controlador). La solución separa claramente el frontend, backend y la base de datos, con una API REST documentada en OpenAPI/Swagger. El proyecto ha seguido la metodología Kanban para visualizar flujos, limitar el WIP y ajustar prioridades en cuatro fases.

Metodológicamente, se partió de un estudio de mercado (Wallapop, Milanuncios, Coches.net, AutoScout24, ehorses, etc.) para identificar buenas prácticas y carencias (filtrado específico, geolocalización, agenda de eventos) y definir la propuesta de valor diferencial.

En la implementación, el backend expone recursos de usuarios, perfiles, anuncios y disciplinas mediante rutas REST y /docs. El frontend consume la API y renderiza listados y fichas con filtros y mapa. El modelo de datos abarca usuarios, sesiones, anuncios, imágenes, disciplinas y revisiones, con restricciones semánticas.

Los resultados acreditan el cumplimiento de los requisitos: registro y gestión de usuarios, ciclo de vida de los anuncios, validación por parte del administrador, búsqueda con filtros y mapa. Quedan pendientes "favoritos", publicidad y gestión de eventos. Se concluye que la arquitectura y el contrato de API facilitan la escalabilidad y la mantenibilidad. Como trabajo futuro, se pretende añadir eventos con calendario y geolocalización, patrocinios, favoritos y notificaciones, así como mejoras de accesibilidad como la integración de nuevos idiomas.

Paraules clau (màxim 10):

Web	Anuncios	Caballos	MVC
JavaScript	MariaDB	Node.js	API
Geolocalización			

ABSTRACT

This work presents the design and implementation of an online commerce platform for horse sales. The objective is to improve the publication, discovery, and management of listings through advanced filters, an interactive map, and a clear MVC (Model - View - Controller) architecture. The solution clearly separates the frontend, backend, and database, with a REST API documented in OpenAPI/Swagger. The project followed the Kanban methodology to visualize workflows, limit WIP, and adjust priorities across four phases.

Methodologically, the project began with a market study (Wallapop, Milanuncios, Coches.net, AutoScout24, ehorses, etc.) to identify best practices and shortcomings (specific filtering, geolocation, event scheduling) and define a distinctive value proposition.

In the implementation, the backend exposes user, profile, listing, and discipline resources via REST routes and /docs. The frontend consumes the API and renders lists and detail pages with filters and a map. The data model includes users, sessions, listings, images, disciplines, and approval records, with semantic constraints.

The results confirm compliance with the main requirements: user registration and management, listing lifecycle, administrator validation, and search with filters and map. "Favorites," advertising, and event management remain pending. It is concluded that the architecture and API contract facilitate scalability and maintainability. Future work aims to include events with calendar and geolocation, sponsorships, favorites, and notifications, as well as accessibility improvements such as multilingual integration.

Keywords (10 maximum):

Web	Listings	Horses	MVC
JavaScript	MariaDB	Node.js	API
Geolocation			

SUMARI

1. INTRODUCCIÓ	11
1.1. OBJECTIU PRINCIPAL	11
1.2. SUBOBJECTIUS	11
1.3. MOTIVACIÓ	12
1.4. ESTUDI DE MERCAT	13
1.4.1. Plataformes Analitzades	13
1.4.2. Conclusions	14
1.5. ESTRUCTURA DEL DOCUMENT	15
2. METODOLOGIA	16
2.1. METODOLOGIA ÀGIL KANBAN	16
2.2. PLANIFICACIÓ	17
Fase 1: Anàlisi del procés (60h)	18
Fase 2: Disseny i desenvolupament de la plataforma (380h)	18
Fase 3: Coordinació (10h)	19
Fase 4: Documentació i presentació del projecte (90h)	19
3. ESPECIFICACIÓ	22
3.1. ACTORS	22
3.2. REQUISITS FUNCIONALS	23
3.3. REQUISITS NO FUNCIONALS	25
3.4. MODEL CONCEPTUAL	26
4. DISSENY	28
4.1. ARQUITECTURA	28
4.2. INTERFICIE (MOCK-UPS)	31
4.3. BASE DE DADES	32
5. IMPLEMENTACIÓ	35
5.1. TECNOLOGIES	35
5.1.1. Eines	35
5.1.2. Tecnologies de desenvolupament	36
5.2. CONFIGURACIÓ DE L'ENTORN DE DESENVOLUPAMENT	37
5.3. CODI	42
5.4. CODI OPEN SOURCE	47
5.5. DOCUMENTACIÓ DE LA API	47
5.6. MANUAL DE L'USUARI	51
5.7. INSTRUCCIONS PER AL DESPLEGAMENT	60
6. VALIDACIÓ	62
6.1. PROVES DE REQUISITS FUNCIONALS	62
6.1.1. Gestió d'usuaris	62
6.1.2. Gestió d'anuncis de venda de cavalls	63
6.1.3. Anuncis destacats (administrador)	64
6.1.4. Cerca i filtres	64
6.1.5. Mapa d'ubicacions	65

6.1.6. Panell de control de l'administrador	65
6.2. ASSOLIMENT DELS REQUISITS NO FUNCIONALS	66
7. SOSTENIBILITAT	69
7.1. MATRIU DE SOSTENIBILITAT	69
7.2. ANÀLISI D'IGUALTAT DE GÈNERE	70
7.3. IMPLICACIONS ÈTIQUES	70
7.4. RELACIÓ AMB ELS OBJECTIUS DE DESENVOLUPAMENT SOSTENIBLE	71
CONCLUSIONS	72
BIBLIOGRAFIA	73

SUMARI DE FIGURES

Figura 1.1	Anàlisi de les funcionalitats de les pàgines web e-commerce i de venda de cavalls _	15
Figura 2.1	Captura de pantalla del funcionament de la metodologia Kanban amb trello _____	17
Figura 2.2	Planificació del projecte _____	20
Figura 2.3	Gràfic de l'execució real _____	21
Figura 3.1	Model conceptual _____	26
Figura 4.1	Exemple d'arquitectura client - servidor _____	28
Figura 4.2	Exemple d'arquitectura MVC _____	29
Figura 4.3	Diagrama de seqüència del cas d'ús Cercar anuncis amb filtres _____	30
Figura 4.4	Mock-up del cercador amb filtres avançats _____	31
Figura 4.5	Mock-up del calendari d'esdeveniments i concursos _____	32
Figura 4.6	Disseny de la base de dades _____	33
Figura 5.1	Endpoints dels usuaris _____	48
Figura 5.2	Endpoint de l'operació registre d'un usuari _____	49
Figura 5.3	Endpoints dels recursos restants del codi _____	50
Figura 5.4	Captura de pantalla de la creació d'un compte _____	51
Figura 5.5	Captura de pantalla de l'inici de sessió _____	52
Figura 5.6	Captura de pantalla de la pàgina d'inici _____	52
Figura 5.7	Captura de pantalla de la pàgina d'anuncis sense filtres _____	53
Figura 5.8	Captura de pantalla de la pàgina d'anuncis amb filtres de disciplines i sexe _____	54
Figura 5.9	Captura de pantalla de la pàgina d'anuncis amb filtres de Capa i preu màxim _____	55
Figura 5.10	Captura de pantalla de la pàgina detalls d'anunci _____	56
Figura 5.11	Captura de pantalla de la secció del perfil d'usuari _____	57
Figura 5.12	Captura de pantalla del formulari de creació d'anuncis _____	58
Figura 5.13	Captura de pantalla de la pàgina de l'administrador _____	59

SUMARI DEL CODI

Codi 5.1	Instal·lació Node.js i npm.	37
Codi 5.2	Instal·lació i posada en marxa de MariaDB.	38
Codi 5.3	Instal·lació de dbeaver.	38
Codi 5.4	Inicialització del projecte Node.js per al backend.	38
Codi 5.5	Instal·lació de dependències del backend.	39
Codi 5.6	Estructura dels directoris del backend.	39
Codi 5.7	Configuració mínima d'Express per arrencar el servidor.	40
Codi 5.8	Variables d'entorn per a la connexió a la base de dades.	40
Codi 5.9	Pool de connexions MariaDB amb mysql2/promise.	40
Codi 5.10	Muntatge de documentació d'API amb Swagger UI.	41
Codi 5.11	Estructura de directoris del frontend.	41
Codi 5.12	Estructura de directoris del frontend.	42
Codi 5.13	Inicialització, publicació i configuració del remot del repositori a GitHub.	42
Codi 5.14	Crida del client a l'endpoint d'anuncis amb els filtres seleccionats.	43
Codi 5.15	Aquest fragment de codi és el responsable de delegar les peticions que es fan a través del fetch.	43
Codi 5.16	Definició de la ruta GET "/" d'anuncis i associació al controlador llistar.	43
Codi 5.17	Controlador llistar: s'encarrega de la lògica, delega els filtres i retorna la resposta.	44
Codi 5.18	Construeix SQL + paràmetres a partir de la query i retorna el resultat.	44
Codi 5.19	Consulta base, enriquiment amb disciplines i normalització de tipus.	45
Codi 5.20	Creació i configuració inicial del mapa.	46
Codi 5.21	Recuperació d'anuncis i renderització de markers i popups al mapa.	47
Codi 5.22	Clonació del repositori.	60
Codi 5.23	Creació de la base de dades i usuari d'aplicació.	60
Codi 5.24	Restauració del dump cavalls_db_dump.sql.	60
Codi 5.25	Instal·lació i arrencada del backend.	61
Codi 6.1	Comentaris/JSDoc als controladors (validació de codi documentat)	67
Codi 6.2	Media queries del disseny responsiu (validació de RNF)	67

GLOSSARI DE SIGNES, SÍMBOLS, ABREVIATURES, ACRÒNIMS I TERMES

- BBDD (Base de Dades): conjunt de dades estructurades gestionades per un SGBD (p. ex., MariaDB).
- SQL (Structured Query Language): llenguatge estàndard per consultar i gestionar BBDD relacionals.
- ACID (Atomicitat, Consistència, Aïllament, Durabilitat): propietats de les transaccions en BBDD.
- MVC (Model-Vista-Controlador): patró que separa dades, presentació i control de flux.
- API (Application Programming Interface): conjunt d'operacions exposades pel backend.
- REST/RESTful: estil d'arquitectura per a APIs basat en recursos i verbs HTTP.
- OpenAPI/Swagger: especificació i UI per documentar i provar APIs.
- WIP (Work In Progress): límit de treball simultani en Kanban per evitar colls d'ampolla.
- Frontend/Backend: capes de client i servidor; el backend implementa la lògica i l'API.
- Geolocalització: obtenció/ús de latitud/longitud per filtrar o mostrar anuncis en mapes.

1. INTRODUCCIÓ

Aquest treball s'emmarca com a Treball de Final de Grau del grau d'Enginyeria Informàtica de l'Escola Politècnica Superior d'Enginyeria de Vilanova i la Geltrú (EPSEVG), centre docent de la Universitat Politècnica de Catalunya (UPC).

En ell es presenta el disseny i la implementació d'una plataforma de comerç online especialitzada en cavalls, orientada a millorar la cerca, la publicació i la validació d'anuncis mitjançant filtres avançats, un mapa interactiu i calendaris d'esdeveniments.

A la introducció es defineix el problema, s'identifiquen les entitats implicades, es concreten els objectius i es delimita l'abast, a més d'explicar l'estructura del document.

1.1. OBJECTIU PRINCIPAL

Aquest treball de final de grau té com a objectiu principal *desenvolupar una plataforma de comerç online especialitzada en la compra i venda de cavalls*, dissenyada per poder cobrir les mancances que podem trobar a les plataformes del sector. Tot i l'existència de plataformes dedicades al món eqüestre, veiem que les funcionalitats que hi ha són limitades, com la manca de geolocalització, la inexistència de calendaris d'esdeveniments per poder veure els cavalls en directe o filtratges d'anuncis molt bàsics.

Aquesta plataforma vol proporcionar una experiència d'usuari optimitzada al sector eqüestre basant-se en els punts forts de plataformes de compra-venda com Wallapop o Coches.net. Permetrà als venedors registrats publicar anuncis detallats de cavalls, mentre que els compradors tindran l'opció de navegar pels anuncis, localitzar els cavalls mitjançant un mapa interactiu i veure els esdeveniments on els cavalls participen en competicions.

Les funcionalitats previstes que destaquen respecte els competidors són la integració d'un mapa interactiu que permet localitzar amb facilitat els cavalls, i la millora de filtres avançats que permeten fer una cerca molt selectiva i ràpida i de mostrar anuncis de cavalls destacats i la integració d'un calendari d'esdeveniments eqüestres amb les dates i ubicacions dels concursos.

Aquest projecte pretén cobrir un buit existent al mercat, aportant una solució adaptada a les necessitats del sector eqüestre.

1.2. SUBOBJECTIUS

Amb l'objectiu principal en ment, es defineixen els següents subobjectius:

- Permetre a l'administrador gestionar els anuncis de cavalls i seleccionar aquells que es volen destacar per augmentar la visibilitat de determinades ofertes.
- Que els usuaris puguin veure clarament la ubicació dels diferents anuncis de cavalls

repartits per tot el territori.

- Mostrar les dates i ubicacions de competicions on els cavalls participen, oferint als compradors l'oportunitat de veure'ls en directe.
- Desenvolupar slots preconfigurats editables per l'administrador, que permetin la publicitat d'empreses o particulars dins la plataforma.
- Desenvolupar una interfície responsive i accessible, adaptada a diferents dispositius.
- Implementar mesures de seguretat per protegir les dades dels usuaris.

1.3. MOTIVACIÓ

He escollit aquest TFG per diverses raons:

- **Nínxol de Mercat Desatès:** tot i que existeixen plataformes de compravenda generalistes, no hi ha cap plataforma especialitzada que abasti de manera eficient i completa el mercat de compra i venda de cavalls. La majoria d'anuncis es troben dispersos en portals molt generalitzats, fet que dificulta tant la visibilitat dels venedors com la cerca per part dels compradors.
- **Funcionalitats Específiques i Innovadores:** la inclusió de funcionalitats com anuncis destacats, mapes d'ubicacions i calendaris d'esdeveniments aporta un valor afegit que actualment no ofereixen les plataformes existents. Això facilita una experiència d'usuari més rica i adaptada a les necessitats del mercat eqüestre.
- **Aprenentatge i Creixement Personal:** el desenvolupament d'aquest projecte permetrà aprofundir en tecnologies web modernes com Express.js per al frontend i Node.js o Django per al backend, així com gestionar una base de dades relacional. Tot i que ja he treballat amb pàgines web, aquest serà el primer cop que crearé des de zero, una plataforma completa de forma autònoma. Fet que suposarà un repte que em permetrà consolidar els meus coneixements en desenvolupament web i gestió de projectes.
- **Impacte en la Comunitat Eqüestre:** la plataforma no només facilitarà les transaccions de compra i venda, sinó que també fomentarà la creació d'una comunitat eqüestre en línia, promovent esdeveniments, competicions i activitats relacionades.
- **Contribució Acadèmica i Professional:** aquest projecte no només compleix els requisits acadèmics per al TFG, sinó que també té un potencial real de ser aplicat en el món professional, demostrant la meua capacitat per desenvolupar solucions digitals a problemes reals.

1.4. ESTUDI DE MERCAT

En aquesta secció s'analitzen plataformes generalistes i especialitzades en venda de cavalls per poder entendre bones pràctiques, funcionalitats clau i punts febles. L'objectiu és detectar oportunitats i veure si la proposta de valor és realment diferencial.

1.4.1. Plataformes Analitzades

Coches.net (<https://www.coches.net>)

Portal especialitzat en la compra i venda de vehicles a Espanya. Disposa de filtres avançats, anuncis destacats, sistema de missatgeria, i validació de publicacions per part de l'equip editorial. Ofereix una experiència molt cuidada a nivell de navegació i cerca.

Wallapop (<https://es.wallapop.com>)

Aplicació de compravenda entre particulars, molt estesa a Espanya, amb sistema de missatgeria intern i aplicació mòbil molt intuïtiva. Ofereix anuncis destacats i possibilitat de pagament dins la plataforma, però amb filtres de cerca molt bàsics.

Milanuncios (<https://www.milanuncios.com>)

Plataforma generalista per comprar i vendre productes de tota mena, inclosos cavalls. Té molts anuncis, però la cerca és molt bàsica, no valida contingut i la interfície és menys moderna.

AutoScout24 (<https://www.autoscout24.es>)

Portal europeu especialitzat en la venda de vehicles, molt complet pel que fa a filtres, validació, anuncis destacats i valoracions. No té sistema de pagament integrat.

ehorses.es (<https://www.ehorses.es>)

Una de les plataformes especialitzades més conegudes a nivell europeu en la venda de cavalls. Inclou filtres per raça i disciplina, però la interfície és limitada i no disposa de mapa interactiu ni calendari d'esdeveniments.

ventadecaballos.es (<https://www.ventadecaballos.es>)

Plataforma espanyola centrada en cavalls i material eqüestre. Tot i estar enfocada al sector, presenta mancances en funcionalitats modernes, com geolocalització, cerca avançada o visualització d'esdeveniments.

1.4.2. Conclusions

Per a fomentar el projecte i detectar oportunitats d'innovació, he realitzat un estudi de mercat que inclou tant plataformes generalistes de compravenda com plataformes especialitzades en cavalls. L'objectiu ha estat identificar bones pràctiques, funcionalitats útils i mancances que la plataforma pugui cobrir amb una proposta de valor més completa i específica per al sector.

Pel que fa a les plataformes generalistes, com Wallapop i Milanuncios, destaca el seu èxit en espais de compravenda entre particulars. Aporten un model senzill, àgil i intuïtiu que ha demostrat ser funcional, i que serveix com a inspiració per facilitar l'accés i la publicació d'anuncis dins aquesta plataforma. Tot i així, la seva manca de filtres específics, geolocalització precisa i verificació d'anuncis fa que no siguin del tot adequades per a productes especialitzats com els cavalls.

Per altra banda, plataformes com Coches.net i AutoScout24 ofereixen un model molt més complet, amb sistemes de cerca avançada, validació d'anuncis, i geolocalització. Aquests portals són una referència clara a l'hora de dissenyar filtres adaptats al món eqüestre (raça, disciplina, edat, preu, ubicació...), així com la possibilitat de destacar certs anuncis dins el catàleg general.

Pel que fa a les plataformes especialitzades en la venda de cavalls, com ehorses i ventadecaballos, tot i que cobreixen parcialment el nínxol, presenten mancances notables que aquest projecte pretén resoldre. En particular, cap de les dues ofereix geolocalització real en un mapa interactiu, ni tampoc un calendari integrat d'esdeveniments eqüestres que permet als compradors saber on i quan poden veure els cavalls en directe. De la mateixa manera, la interfície d'usuari i l'experiència general són poc modernes, amb estructures poc optimitzades per a mòbils. Tampoc no s'ofereix una gestió d'anuncis validada per administradors ni espais per a publicitat personalitzada.

✓ = disponible | ✗ = no disponible | ~ = parcial / limitat

Funcionalitat	Wallapop	Milanuncios	Coches.net	AutoScout24	ehorses	ventadecaballos	Plataforma Cavalls
Filtres específics	✗	✗	✓	✓	✓	~	✓
Mapa interactiu	~	~	✓	✓	✗	✗	✓
Cerca per distància	✓	✓	✓	✓	✓	~	✓
Validació editorial/admin dels anuncis	✗	✗	✓	✓	~	✗	✓
Anuncis destacats / promocionats	✓	✓	✓	✓	✓	~	✓
Calendari d'esdeveniments integrat	✗	✗	✗	✗	✗	✗	✓
App mòbil	✓	~	✓	✓	~	~	✗
Pagament integrat	✓	~	✗	✗	✗	✗	✗
Galeria amb fotos	✓	✓	✓	✓	✓	✓	✓
Gestió d'anuncis (editar/pausar/estadístiques)	~	~	✓	✓	✓	~	✓

Figura 1.1. Anàlisi de les funcionalitats de les pàgines web e-commerce i de venda de cavalls. Font: creació pròpia.

1.5. ESTRUCTURA DEL DOCUMENT

Capítol 1 - Introducció: context, objectius, abast i estructura del treball.

Capítol 2 - Metodologia: Kanban, planificació i fases del projecte.

Capítol 3 - Requisits: actors, requisits funcionals/no funcionals i model conceptual.

Capítol 4 - Disseny: arquitectura, patrons, mock-ups i esquema de dades.

Capítol 5 - Implementació: tecnologies, entorn, codi, documentació d'API, manual i desplegament.

Capítol 6 - Validació: proves d'acceptació i comprovació de requisits.

Capítol 7 - Sostenibilitat: impacte, igualtat de gènere, ètica i ODS.

2. METODOLOGIA

Aquest apartat descriu el marc de treball emprat per gestionar i executar el projecte: s'explica l'elecció de la metodologia àgil Kanban, i la seva aplicació pràctica, la distribució en 4 fases del projecte i la planificació temporal de tasques juntament amb un gràfic per entendre visualment l'abast del pla de treball, així com el procés d'adaptació contínua davant canvis i imprevistos.

2.1. METODOLOGIA ÀGIL KANBAN

Per al desenvolupament d'aquest projecte s'ha optat per la metodologia Àgil, concretament el sistema Kanban [1]. Aquesta metodologia es caracteritza per oferir una gestió visual del flux de treball i per prioritzar la flexibilitat i l'adaptació constant als canvis del projecte. A diferència d'altres mètodes àgils, com Scrum [2], Kanban no estableix iteracions fixes, sinó que es basa en un flux continu de treball. Aquesta elecció respon al fet que el desenvolupament és individual i que les funcionalitats del projecte estaven definides des de l'inici. Així, no és necessari establir sprints, i la única iteració formal són les reunions bisetmanals amb els directors per revisar l'estat del projecte i ajustar prioritats.

Els principis generals de Kanban que s'aplicaran en aquest projecte són els següents:

1. Visualitzar el flux de treball

La clau del sistema Kanban és la representació visual de les tasques i del seu estat. Per aquest projecte s'utilitzarà un taulell Kanban amb quatre columnes principals:

- Backlog: Tasques identificades però encara no prioritzades per al desenvolupament.
- To Do: Tasques planificades i pendents d'iniciar.
- Doing: Tasques en desenvolupament actiu.
- Done: Tasques completades.

Aquesta representació permet un seguiment clar i transparent de l'avanç del projecte.

2. Límits de Work In Progress (WIP)

Un dels objectius principals és evitar colls d'ampolla i dispersió de recursos. Per això, s'establirà un límit de treball en curs que restringirà el nombre de tasques que poden estar a la columna Doing de manera simultània. Aquesta mesura permet assegurar un flux constant i sostenible.

3. Flux de treball continu

A diferència de Scrum, que es basa en sprints tancats, Kanban promou un flux de treball continu. Les tasques s'afegeixen al taulell segons la seva prioritat i es gestionen de manera dinàmica, sense dependència d'iteracions fixes. Això permet una millor adaptabilitat als canvis i facilita l'optimització del temps.

4. Reunions de seguiment

Tot i que Kanban no contempla reunions formals, en aquest projecte s'han establert reunions de seguiment cada dues setmanes amb els directors. En aquestes reunions es revisa l'estat de les tasques, es recull feedback i s'ajusten les prioritats del desenvolupament. Aquest procés garanteix una millora contínua i una adaptació ràpida als requeriments del projecte.

5. Adaptabilitat constant

La metodologia Kanban ofereix la possibilitat d'incorporar noves funcionalitats o modificar les existents en qualsevol moment, sense que això suposi una replanificació completa. Aquesta flexibilitat és clau per a un projecte individual i acadèmic, en què les necessitats poden variar lleugerament segons les indicacions dels directors.

A la següent figura es mostra el taulell Kanban del projecte implementat amb l'eina Trello. En aquest taulell es poden veure les diferents columnes (Backlog, To Do, Doing, Done), així com les targetes corresponents a cada tasca del desenvolupament. Aquesta representació visual permet fer un seguiment clar i organitzat de l'estat de les activitats del projecte i facilita la gestió del flux de treball de manera contínua.

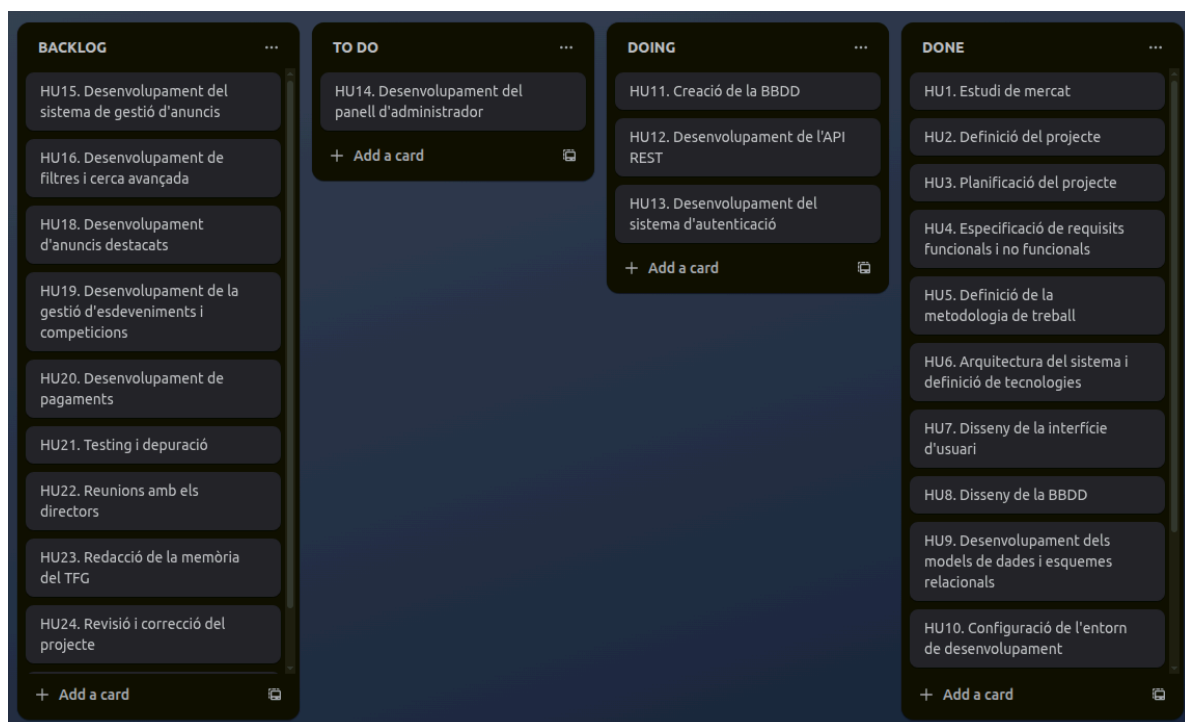


Figura 2.1. Captura de pantalla del funcionament de la metodologia Kanban amb trello. Font: elaboració pròpia amb Trello [\[3\]](#)

2.2. PLANIFICACIÓ

Per a la planificació he dividit el projecte en 4 fases. I he assignat diverses tasques a cada una d'aquestes fases.

Fase 1: Anàlisi del procés (60h)

En aquesta fase es recollirà informació sobre plataformes similars, es definirà l'objectiu del projecte, els seus sub-objectius i la metodologia de treball.

- **T1. Estudi del mercat (10h):** Anàlisi de plataformes similars per identificar funcionalitats útils i diferenciació de la proposta de valor.
- **T2. Definició del projecte (8h):** Redacció de l'objectiu, justificació i abast del projecte, establint requisits generals.
- **T3. Planificació del projecte (10h):** Creació de la planificació temporal amb diagrama de Gantt i metodologia àgil.
- **T4. Especificació de requisits funcionals i no funcionals (12h):** Definició detallada de la gestió d'usuaris, anuncis, cerca, seguretat, accessibilitat i internacionalització.
- **T5. Definició de la metodologia de treball (10h):** Selecció d'un enfocament àgil (Kanban), eines de gestió i documentació.
- **T6. Arquitectura del sistema i selecció de tecnologies (10h):** Comparació entre alternatives i elecció de la més adequada.

Fase 2: Disseny i desenvolupament de la plataforma (380h)

En aquesta fase es dissenyarà l'arquitectura de l'aplicació i es desenvoluparan les funcionalitats principals.

- **T7. Disseny de la interfície d'usuari (10h):** Creació de wireframes i prototips interactius per a les principals pantalles.
- **T8. Disseny de la base de dades (30h):** Creació del diagrama esquema relacional.
- **T9. Desenvolupament dels models de dades i esquemes relacionals (20h):** Creació dels models en SQL i relacions entre entitats.
- **T10. Configuració de l'entorn de desenvolupament (15h):** Instal·lació i configuració d'eines (Node.js, Express.js, MariaDB, Github).
- **T11. Creació de la base de dades (5h):** Càrrega de dades fictícies.
- **T12. Desenvolupament de l'API REST (40h):** Desenvolupament dels endpoints per a la gestió d'usuaris, autenticació i anuncis.
- **T13. Desenvolupament del sistema d'autenticació (30h):** CRUD complet dels 2 tipus d'usuaris (administradors i usuaris).
- **T14. Desenvolupament del panell d'administrador (30h):** Creació d'un panell per gestionar anuncis destacats i usuaris.
- **T15. Desenvolupament del sistema de gestió d'anuncis (30h):** CRUD complet d'anuncis amb imatges i descripció.
- **T16. Desenvolupament de filtres i cerca avançada (30h):** Creació de filtres per raça, preu, ubicació, etc.

- **T17. Desenvolupament del mapa d'ubicacions (35h):** Integració amb OpenStreetMap per mostrar la ubicació dels anuncis.
- **T18. Desenvolupament de publicitat destacada (30h):** Desenvolupament del sistema per promocionar publicitat segons configuració de l'administrador.
- **T19. Desenvolupament de la gestió d'esdeveniments i competicions (35h):** Creació del calendari d'esdeveniments amb funcionalitat de cerca i subscripció.
- **T20. Testing i depuració (40h):** Proves de funcionalitat i correcció d'errors.

Fase 3: Coordinació (10h)

Aquesta fase se centrarà a validar el correcte funcionament de la plataforma, corregint errors i millorant la seva usabilitat.

- **T21. Reunions amb els directors (10h):** Es manté una reunió cada 2 setmanes per fer un seguiment i poder corregir el projecte.

Fase 4: Documentació i presentació del projecte (90h)

En aquesta última fase es prepararà la memòria final i la defensa oral.

- **T22. Redacció de la memòria del TFG (40h):** Integració de documentació prèvia i seccions finals.
- **T23. Revisió i correcció del projecte (20h):** Revisió final del codi, documentació i proves.
- **T24. Preparació de la presentació final (30h):** Creació del material de suport per a la defensa oral i pràctica de l'exposició.

Per a poder fer una planificació correcta, he utilitzat un gràfic de Gantt [\[4\]](#), una eina que em permet visualitzar en el temps la distribució de les diferents tasques.

Aquesta planificació està pensada per a distribuir les 540 hores del projecte en 20 hores de treball setmanal, tenint en compte que a l'agost no es treballarà.

Plataforma de Comerç Online per a la Venda de Cavalls

Francesco Oncins Spedo

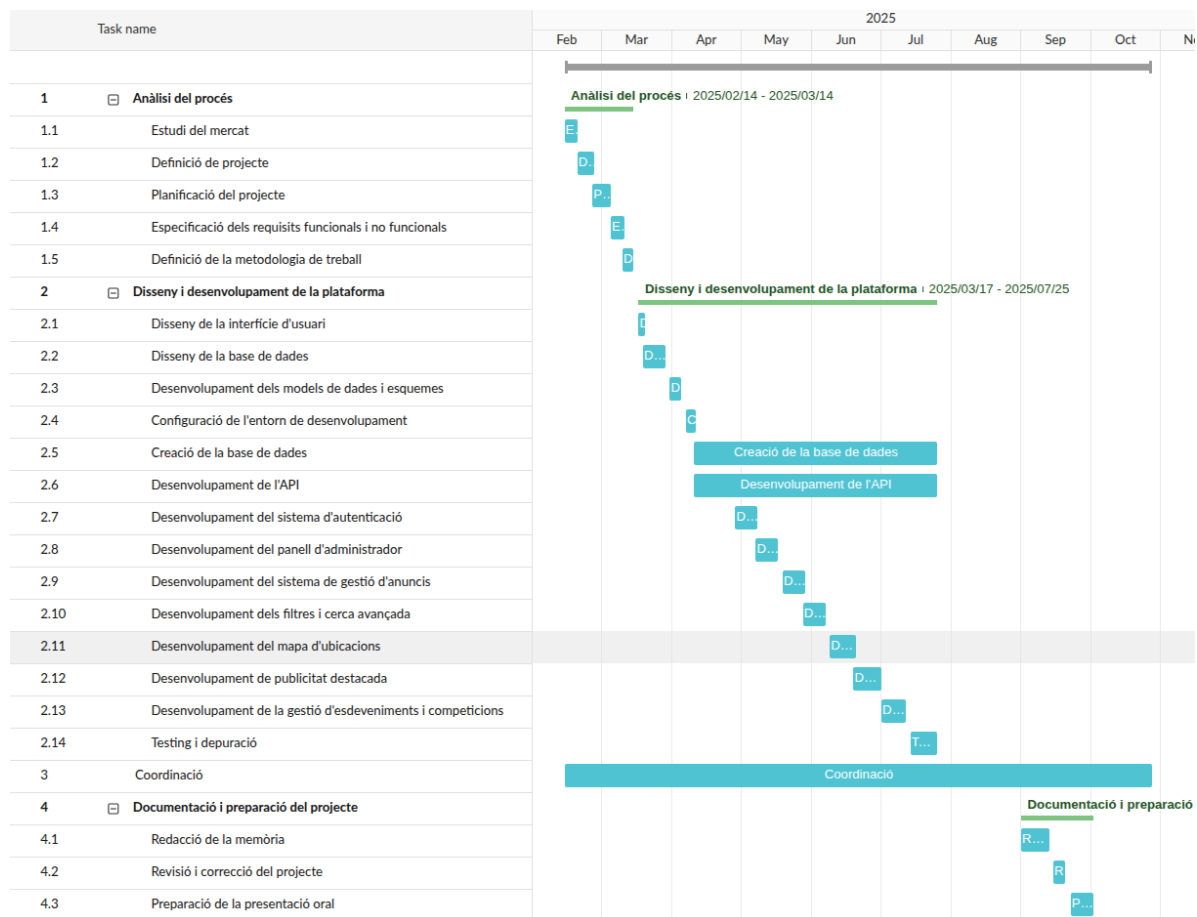


Figura 2.2. Planificació del projecte. Font: elaboració pròpia amb Gantt [4]

Durant l'execució del projecte no he pogut seguir estrictament la planificació inicial perquè diverses tasques han requerit més dedicació de la prevista. En concret, la T11. Creació de la base de dades, la T13 Desenvolupament del sistema d'autenticació, la T14. Panell d'administrador, la T15. Gestió d'anuncis, la T16. Filtres i cerca avançada i la T17. Mapa d'ubicacions han esdevingut més complexes del que s'havia plantejat prèviament i per tant han necessitat més hores per a ser completades.

A la T11, la creació de taules, la normalització de dades i la càrrega de dades de prova han requerit 5 hores addicionals a les previstes.

De la tasca T13, Desenvolupament del sistema d'autenticació, no havia planificat la creació d'un sistema de sessions, que vaig haver de crear per a poder gestionar l'inici i el tancament de les sessions dels usuaris.

Pel que fa a les tasques T14, T15, T16 i T17, senzillament he hagut de dedicar més temps del previst a funcionalitats com la responsivitat en pantalles de diferents mides, la lògica per a poder guardar diverses imatges dels anuncis, o poder aplicar un filtratge complet dels anuncis.

Com a conseqüència, s'han hagut de posposar les tasques T18. Publicitat destacada i

T19. Gestió d'esdeveniments i competicions, ja que s'ha considerat que la publicitat no era crítica en aquesta fase, i que la gestió d'esdeveniments exigia més temps del previst per a poder fer-se bé. Aquesta decisió ha permès entregar una versió més sòlida de les funcionalitats principals, mantenint un focus en la qualitat, mantenibilitat i funcionalitats bàsiques, assumint com a treball futur la incorporació de la publicitat i els esdeveniments.

Degut a aquests canvis a la planificació he fet un Gantt que mostra la planificació real:

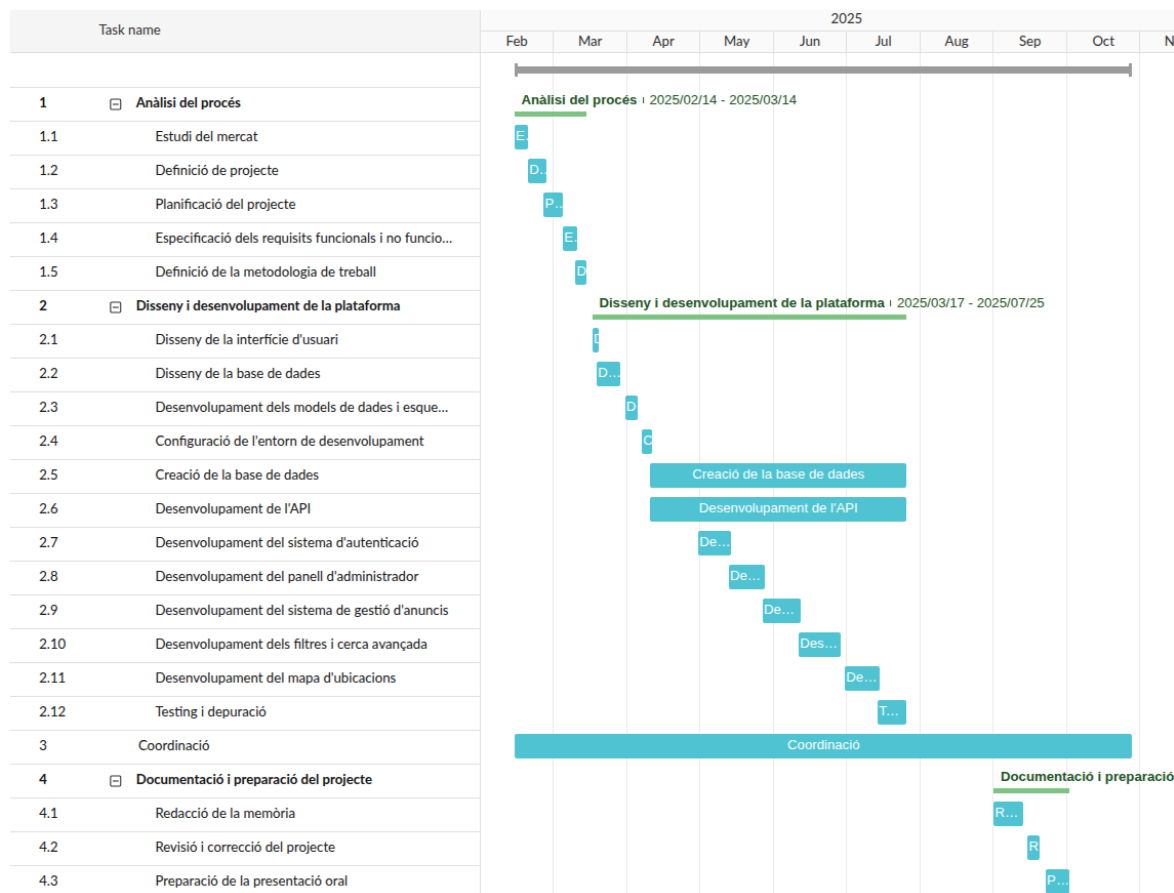


Figura 2.3. Gràfic de l'execució real. Font: elaboració pròpia amb Gantt [4]

3. ESPECIFICACIÓ

Aquesta secció defineix l'abast i el comportament del sistema. S'hi identifiquen els actors i els seus rols, es detallen els requisits funcionals i els no funcionals. Es completa amb el model conceptual i les restriccions semàntiques que acoten el domini de dades, així com els supòsits i decisions que guien la verificació i validació de la plataforma.

3.1. ACTORS

En el sistema de plataforma de comerç online per a la venda de cavalls, es defineixen els següents actors principals:

Actors principals

Administrador

L'administrador és l'actor responsable de la gestió i supervisió del sistema. Té els permisos més amplis i pot accedir a totes les funcionalitats de gestió i configuració de la plataforma. Les seves responsabilitats inclouen la gestió d'usuaris, amb tot el que això comporta, la validació d'anuncis i la selecció dels més importants i la gestió dels slots publicitaris. També s'encarrega de moderar el contingut de la plataforma, pot consultar i generar informes i estadístiques i configurar el sistema.

Usuari

A la plataforma hi ha un únic tipus d'usuari, que pot actuar com a comprador o com a venedor segons les seves necessitats.

L'usuari pot navegar lliurement per la web sense necessitat de registrar-se, consultant els anuncis disponibles i tota la informació dels cavalls sense cap restricció. Tot i això, per poder publicar anuncis de venda de cavalls o gestionar-los, és necessari estar registrat.

Quan l'usuari es registra, adquireix la possibilitat de crear i administrar els seus propis anuncis amb informació detallada dels cavalls, així com programar esdeveniments on hi participin, indicant la data i el lloc perquè els compradors puguin veure'ls en directe.

Tant els usuaris registrats com els no registrats poden veure els anuncis destacats i els anuncis publicitaris que es mostren a la plataforma.

Stakeholders

Jordi García (Director del TFG). Responsable de supervisar l'avenç del projecte, validar les decisions tècniques i proporcionar orientació acadèmica.

Lídia López (Codirectora del TFG). Ofereix suport en la gestió del projecte i revisió dels aspectes tècnics i metodològics.

Francesco Oncins (Desenvolupador i responsable del projecte). Responsable del desenvolupament i implementació del sistema, així com de la documentació, presentació i validació del TFG.

3.2. REQUISITS FUNCIONALS

En aquesta secció es detallen les diferents funcionalitats que s'ofereixen als usuaris de la plataforma de comerç online per a la venda de cavalls, agrupades en diferents categories.

Per a cada requisit s'indica si s'ha completat o no en aquesta primera versió.

Gestió d'usuaris

- **Alta d'usuaris.** Completat
Permetre als usuaris registrar-se per publicar anuncis de cavalls i que els potencials compradors puguin guardar-los i posar-se en contacte.
- **Modificació d'usuaris.** Completat
Els usuaris podran editar les seves dades personals i preferències de notifikacions.
- **Baixa d'usuaris.** Completat
Els usuaris registrats podran eliminar el seu compte del sistema.
- **Autenticació i autorització d'usuaris.** Completat
El sistema verificarà les credencials d'inici de sessió i assignarà permisos segons el rol de l'usuari (Usuari, Administrador).

Panell de control de l'administrador

- **Gestió d'usuaris.** Completat
Administrar altes, baixes i modificacions d'usuaris, incloent l'activació o desactivació de comptes.
- **Validació d'anuncis de cavalls.** Completat
Els anuncis creats o modificats seran revisats i aprovats per l'administrador abans de ser publicats.
- **Informes i estadístiques.** Incomplet
Generar informes sobre vendes, visualitzacions d'anuncis i altres mètriques rellevants.
- **Llista de rebutjats i pendants de validar.** Completat
L'administrador ha de validar un anunci abans que sigui visible a la plataforma.

Gestió d'anuncis de venda de cavalls

- **Alta d'anuncis.** Completat
Els usuaris podran crear anuncis de cavalls amb informació detallada com la raça,

l'edat, el preu, la ubicació i imatges. (ha d'aparèixer l'estat) (hi ha femelles castrades)

- **Modificació d'anuncis.** Completat

Els usuaris podran editar la informació dels seus anuncis de cavalls en qualsevol moment.

- **Baixa d'anuncis.** Completat

Els usuaris podran donar de baixa els seus anuncis quan el cavall hagi estat venut o ja no estigui disponible. Quan un cavall estigui venut, el propietari marcarà la casella corresponent i només el comprador podrà veure els detalls del cavall comprat. De manera que un cavall

- **Visualització d'anuncis.** Completat

Els usuaris (registrats o no) podran veure els anuncis amb informació detallada dels cavalls en venda.

- **Anuncis preferits.** Incomplet

Els usuaris registrats podran guardar els seus anuncis preferits.

Cerca i filtres

- **Filtres avançats de cerca.** Completat

Els usuaris podran cercar cavalls mitjançant filtres com raça, edat, preu, ubicació (en un mapa), i disponibilitat en esdeveniments (en un calendari).

Mapa d'ubicacions

- **Visualització de mapa.** Completat

Els usuaris podran veure en un mapa interactiu les ubicacions dels cavalls en venda.

- **Cerca per proximitat.** Completat

Cercar cavalls en venda per proximitat geogràfica utilitzant el mapa interactiu.

Gestió d'esdeveniments d'anuncis

- **Creació d'esdeveniments.** Incomplet

Els venedors podran programar esdeveniments o competicions on participen els seus cavalls, especificant la data i el lloc.

- **Consulta d'esdeveniments.** Incomplet

Els compradors podran consultar el calendari d'esdeveniments per veure els cavalls en directe.

- **Modificació i cancel·lació d'esdeveniments.** Incomplet

Els venedors podran editar o cancel·lar esdeveniments publicats anteriorment.

Publicitat destacada

- **Gestió d'slots publicitaris.** Incomplet
L'administrador podrà gestionar i editar els slots preconfigurats per a anuncis d'empreses o persones que paguen per aparèixer a la pàgina web.
- **Visualització de publicitat.** Incomplet
La pàgina principal mostrarà anuncis destacats i publicitat de patrocinadors en els slots configurats.

3.3. REQUISITS NO FUNCIONALS

A més dels requisits funcionals, el sistema ha de complir amb una sèrie de requisits no funcionals que garanteixin una experiència d'usuari òptima, un alt nivell de seguretat i un rendiment eficient.

Usabilitat

- **Interfície intuïtiva i fàcil d'utilitzar:** el sistema ha de proporcionar una experiència d'usuari fluida i intuïtiva, amb una navegació senzilla i formularis clars.
- **Disseny responsiu:** la interfície d'usuari ha de ser adaptable a diferents dispositius (ordinadors, mòbils, tauletes).

Rendiment i eficiència

- **Temps de resposta ràpid:** el sistema ha de respondre ràpidament a les accions dels usuaris, garantint una experiència fluida.
- **Escalabilitat:** la plataforma ha de ser capaç de gestionar un augment en el nombre d'usuaris i anuncis sense que es vegi afectada la seva velocitat o eficàcia.
- **Optimització de consultes a la base de dades:** les consultes han d'estar optimitzades per evitar colls d'ampolla en el rendiment.

Seguretat

- **Autenticació i autorització:** implementar un sistema robust d'autenticació i control d'accés per protegir les dades dels usuaris.

Mantenibilitat i extensibilitat

- **Codi modular i documentat:** el codi font ha de ser modular i ben documentat per facilitar el manteniment i la futura extensió de funcionalitats.
- **Fiabilitat:** el sistema ha de funcionar sense errors.

Internacionalització

- **Suport per a diferents idiomes:** la plataforma ha de suportar múltiples idiomes, permetent als usuaris seleccionar la seva llengua preferida.

3.4. MODEL CONCEPTUAL

Aquest model conceptual es va crear a l'inici del projecte el que implica que representa les dades que cobreixen tots els requisits funcionals. Això vol dir que en aquesta primera versió de la Plataforma de Comerç Online per a la Venda de Cavalls, no s'han de tenir en compte les classes Publicitat, Imatge Publicitat i Esdeveniment, ja que com ha estat comentat prèviament aquestes funcionalitats no han estat aplicades. Tampoc s'ha aplicat la relació Favorit entre Anunci i Usuari, que permet que un usuari guardi els seus anuncis preferits.

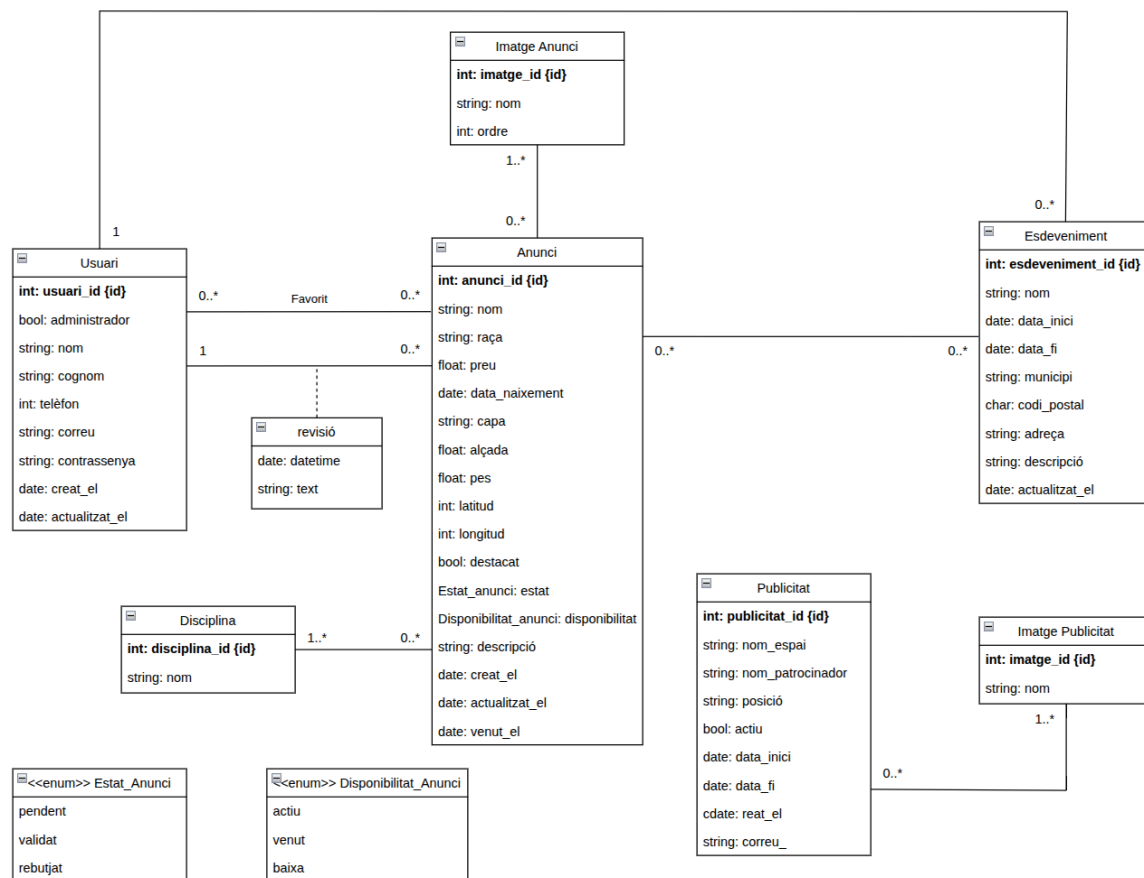


Figura 3.1. Model conceptual. Font: elaboració pròpia amb draw.io [5]

Restriccions Semàntiques (RS)

RS1 (u: Usuari): u.correu i e.telèfon són únics

RS2 (r: revisió): l'Usuari u de r té u.administrador == true

RS5 (a: Anunci): per defecte a.estat = pendent

RS6 (a: Anunci): si a.latitud == NULL a.longitud també ho serà i vice-versa

RS7 (a: Anunci): a.preu > 0, a.alçada > 0 i a.pes > 0

RS8 (ia1, ia2: ImatgeAnunci): ia1.ordre no pot ser igual a ia2.ordre si ia1 i ia2 pertanyen al mateix anunci

RS9 (e: Esdeveniment): e.codi_postal té una longitud de 5 caràcters, on cada caràcter és un dígit (0..9)

RS10 (e: Esdeveniment): e.data_fi >= e.data_inici

RS11 (d: Disciplina): d.nom es correspon a una de les 14 disciplines eqüestres reconegudes per la Real Federación Hípica Española

RS12 (p: Publicitat): p.data_fi >= p.data_inici

RS13 (p: Publicitat): p.correu és únic

4. DISSENY

Aquest capítol descriu les decisions de disseny de la plataforma: s'hi justifica l'arquitectura client-servidor i el patró MVC, definint clarament les responsabilitats de frontend, backend i base de dades. S'hi il·lustra el recorregut complet d'una petició amb un diagrama de seqüència del cas "Cercar anuncis amb filtres" per mostrar com funciona el MVC. Es presenten els mock-ups de les pantalles clau i els principis d'UI/UX que els guien; i es detalla el model de dades a MariaDB (taules, relacions, índexs i integritat). L'objectiu és garantir modularitat, escalabilitat, mantenibilitat i seguretat com a base de l'evolució del sistema.

4.1. ARQUITECTURA

La arquitectura que he adoptat és la de client - servidor separant de manera clara les diferents responsabilitats:

- **Frontend:** responsabilitat d'interacció amb l'usuari (UI/UX), navegació i gestió d'estat al client.
- **Backend:** responsabilitat de lògica de negoci, validacions, seguretat, integració amb serveis externs i accés a dades.
- **BBDD:** responsabilitat de persistència, integritat i consultes eficients (índexs i vistes).

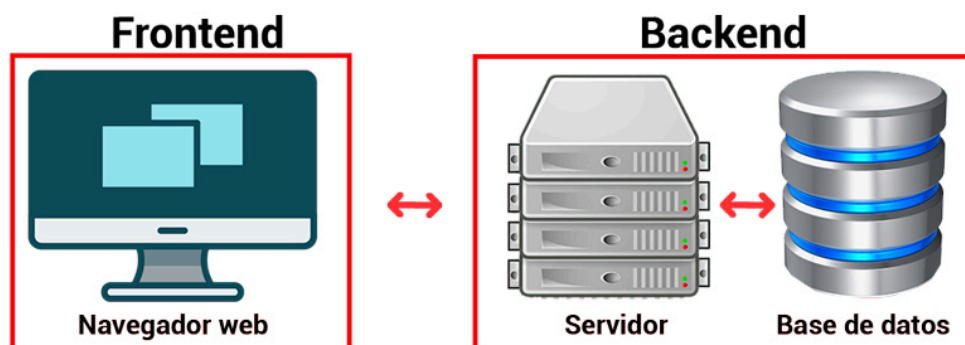


Figura 4.1. Exemple d'arquitectura client - servidor. Font: [\[6\]](#)

Pel que fa al patró arquitectònic que he escollit, tot hi que hi ha moltes opcions, he valorat 2 alternatives entre el patró de tres capes i el de model view controller. Però estudiant les funcionalitats de cada patró em vaig decidir pel model view controller degut a què és un patró dissenyat per a pàgines web i interfícies d'usuari.

Les funcionalitats dels 2 patrons són les següents:

- **Tres Capes** (Presentació - Negoci - Dades): separa responsabilitats a nivell lògic. És excel·lent per a desplegament i per tenir un contracte net entre UI/negoci/dades. Però no especifica com estructurar la presentació o els fluxos interns dins del backend.
- **Model View Controller (MVC)**: és un patró d'estructura interna orientat al flux de peticions/respostes i a la separació Controlador - Model - Vista. Aporta claredat en casos d'ús, rutes i validacions.

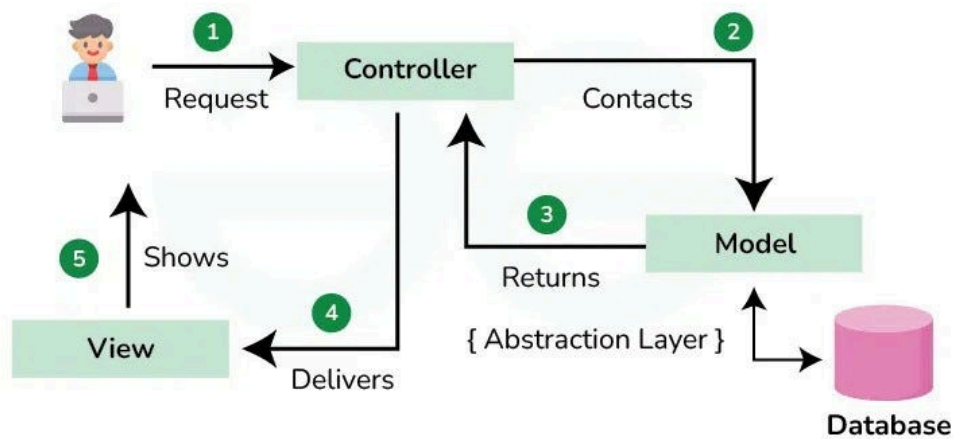


Figura 4.2. Exemple d'arquitectura MVC. Font: [7]

L'arquitectura MVC adoptada en aquest projecte es compon de tres elements principals:

- **Model**. El Model representa les dades i la lògica associada a la seva gestió. En aquest projecte, els models defineixen les entitats del sistema (com Usuari, Anunci, Cavall o Esdeveniment), i especifiquen com s'emmagatzemen i relacionen dins la base de dades MySQL.
També inclouen les validacions de dades i les regles d'integritat. A més, a través de consultes SQL directes, el model s'encarrega de la comunicació amb la base de dades i de transformar les dades a un format adequat perquè el controlador les pugui utilitzar.
- **View**. La View és la capa responsable de la presentació i la interacció amb l'usuari. En aquest cas, està completament desenvolupada estàticament amb html i js, que permet la creació d'una interfície dinàmica i modular. Les vistes consumeixen les dades proporcionades pel controlador a través de l'API REST i les mostren de manera atractiva i responsive.
- **Controller**. El Controller actua com a intermediari entre la Vista i el Model. És l'encarregat de rebre les peticions HTTP del client, validar-les, cridar els models corresponents per obtenir o modificar dades i finalment retornar la resposta en format JSON al frontend.
A més, conté part de la lògica de negoci, gestiona errors, controla l'autenticació dels usuaris i garanteix la seguretat de les operacions mitjançant middleware d'Express. Aquest enfocament permet mantenir el codi modular, clar i fàcilment escalable, facilitant la incorporació de noves funcionalitats o serveis al futur.

A continuació s'ha elaborat un diagrama per visualitzar el recorregut complet d'una petició en el cas d'ús Cercar anuncis amb filtres, evidenciant l'aplicació del patró MVC en la meua plataforma. L'objectiu és mostrar com una acció iniciada per l'usuari es propaga capa a capa (Vista - Controlador - Model - BBDD) i retorna fins a la interfície amb un resultat ordenat, complint les precondicions i postcondicions del cas d'ús.

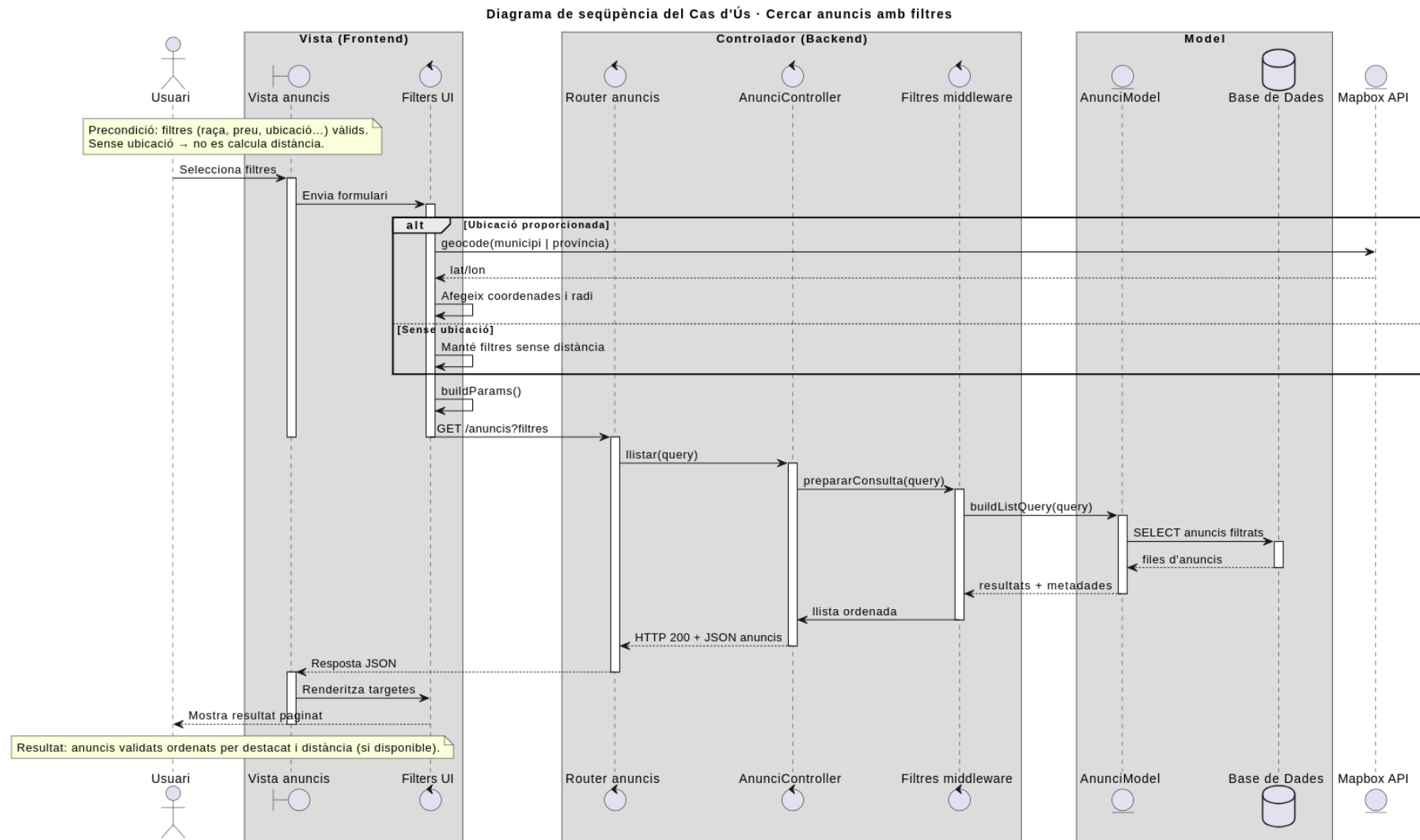


Figura 4.3. Diagrama de seqüència del cas d'ús Cercar anuncis amb filtres. Font: elaboració pròpia amb PlantUML [8]

L'usuari envia els criteris de filtre des de la vista del frontend, que delega al helper de la interfície de filtres. Si hi ha dades d'ubicació, la interfície crida Mapbox per obtenir la latitud i longitud, enriqueix la petició i construeix la consulta completa. El navegador crida el router del backend, que envia al controlador i al middleware de filtres per preparar l'SQL a través de la capa de model i executar-lo sobre MySQL. Els anuncis filtrats tornen a través del controlador i del router, retornant una resposta JSON. La vista renderitza els anuncis amb l'ajuda del helper, presentant a l'usuari els resultats ordenats i paginats.

4.2. INTERFÍCIE (MOCK-UPS)

Els mockups serveixen per validar amb antelació el flux d'usuari, la jerarquia visual i els components clau de la plataforma abans d'entrar en la implementació. En el meu cas, els he dissenyat per prioritzar: la cerca eficient d'anuncis mitjançant filtres i un mapa, la visibilitat d'anuncis destacats i la futura integració d'esdeveniments eqüestres. Com que l'objectiu principal dels mock-ups és tenir una visió general de com serà la versió final de la pàgina web, no he tingut en compte decisions purament d'estil.

Per a crear-los m'he basat en una sèrie de principis que he considerat essencials:

- Claredat: l'usuari veu primer el cercador i els filtres; els resultats ocupen el màxim espai disponible.
- Consistència: targetes d'anunci homogènies (imatge, atributs, estat, accions).
- Visibilitat: tipografia neta, llistes i etiquetes curtes, preu i ubicació visibles.
- Accessibilitat: camps etiquetats, contrast suficient, focus visible i navegació amb teclat.

Pantalla del Cercador

L'objectiu és buscar i comparar cavalls ràpidament amb filtres avançats i, quan l'usuari ho vulgui, acotar per proximitat des d'un mapa.

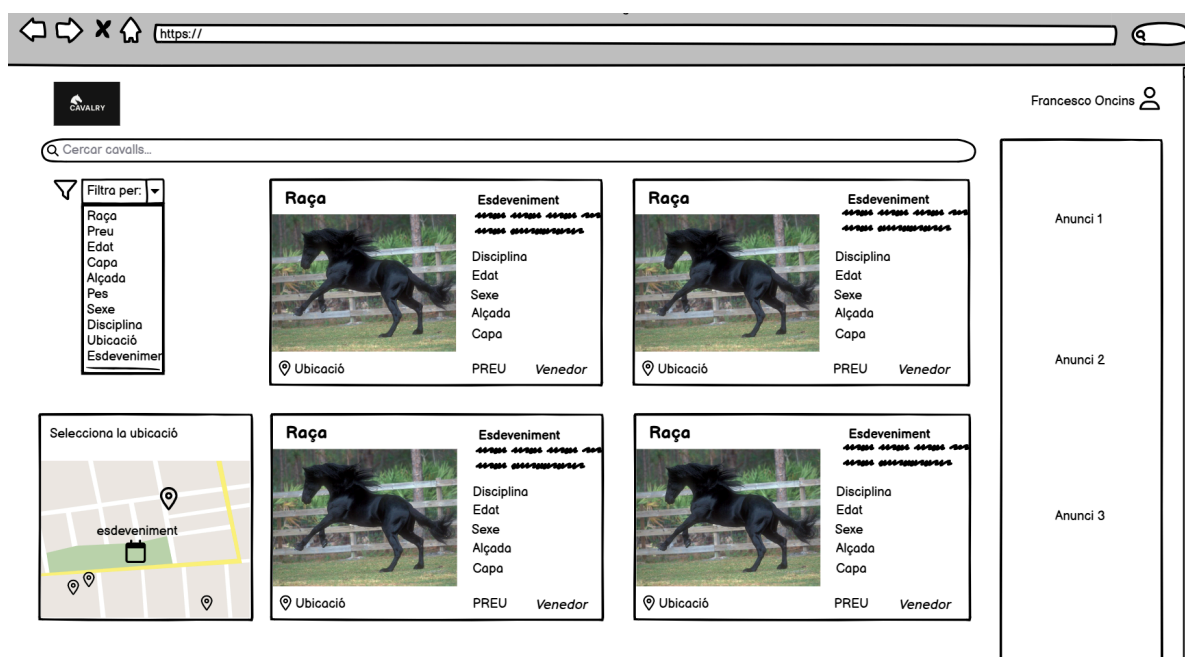


Figura 4.4. Mock up de la pantalla del cercador. Font: elaboració pròpia amb balsamiq [9]

Pantalla d'Esdeveniments

L'objectiu és oferir una agenda eqüestre amb un calendari i un llistat ordenat per data/ubicació, i vincular anuncis de cavalls que es puguin veure en directe en aquell esdeveniment.

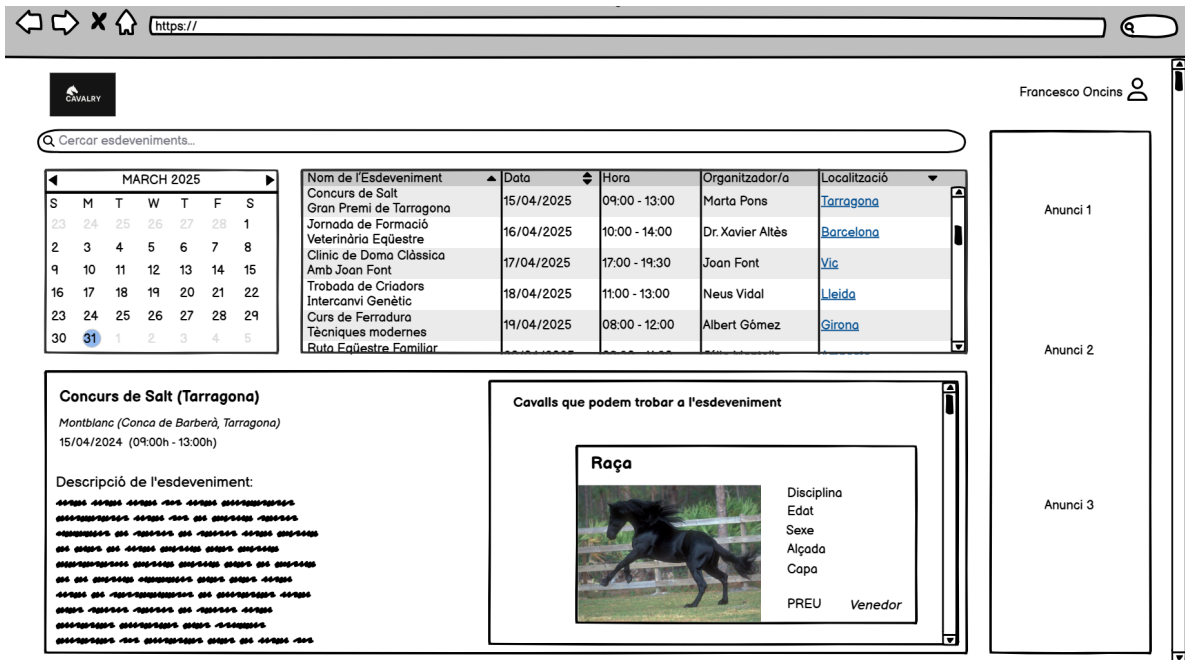


Figura 4.5. Mock up de la pantalla d'esdeveniments. Font: elaboració pròpia amb balsamiq [9]

4.3. BASE DE DADES

Per a dissenyar la base de dades, es va fer un estudi de les diferents opcions a implementar. Primer de tot es va escollir utilitzar una base de dades relacional (SQL). Perquè permet definir un model basat en taules amb claus primàries i foranes i treballar amb transaccions que segueixen les propietats ACID (Atomicitat, Consistència, Aïllament i Durabilitat).

Amb les bases de dades no relacionals descartades es van considerar 3 opcions: SQLite, PostgreSQL i MariaDB.

1. SQLite

És un motor molt bàsic, i per tant fàcil d'utilitzar. No necessita d'un servidor i funciona en un sol fitxer.
Aquesta simplicitat implica limitacions de concurrència i d'administració, necessàries per al correcte desenvolupament del projecte.

2. PostgreSQL

Dels 3, és el motor més complet i robust, permetent funcionalitats avançades i una molt bona escalabilitat.
Adoptar aquest sistema hagués implicat una corba d'aprenentatge major, fent que aquesta segona opció tampoc fos viable.

3. MariaDB

Ofereix un equilibri convenient. És compatible amb MySQL, és robust i escalable, i inclou tot el que el projecte necessita (transaccions, claus foranes, índexs compostos, entre altres).

A més, el fet que MySQL sigui el motor principal amb el que s'ha treballat a la facultat facilita la posada en marxa de la base de dades.

Per aquests motius, la decisió ha estat utilitzar MariaDB.

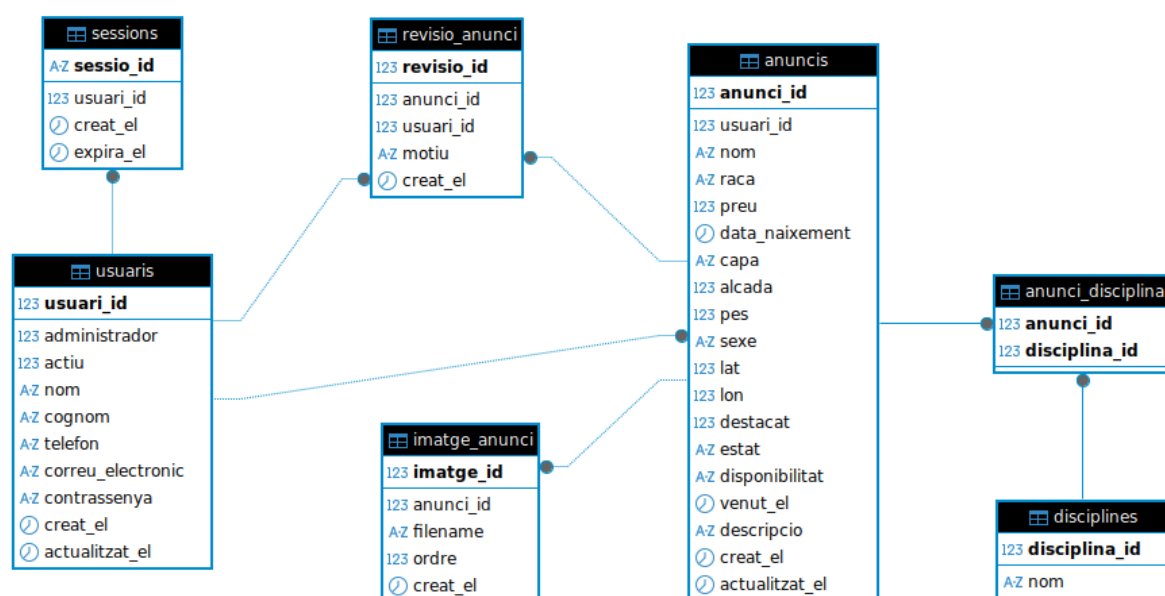


Figura 4.6. Disseny de la base de dades. Font: elaboració pròpia amb dbeaver [10]

Per garantir un producte mínim viable fàcil de mantenir, el disseny es centra en les peces estrictament necessàries per publicar, moderar i cercar anuncis..

La taula usuarios agrupa el perfil i les credencials, amb camps per al rol d'administrador i l'estat del compte (alta/baixa). Aquesta informació es complementa amb sessions, que registra les connexions actives, millorant la seguretat i l'experiència d'usuari (tancament de sessió, expiració, etc.).

Els anuncis, recullen totes les propietats rellevants del cavall (raça, preu, alçada, pes, capa, data de naixement,...). La visibilitat pública ve determinada per l'estat (pendent, aprovat, rebutjat) la qual cosa facilita els fluxos d'aprovació i la consistència del que es mostra a la plataforma. Addicionalment, cada anunci disposa d'un camp de disponibilitat (actiu, venut, baixa) que reflecteix la situació comercial: "actiu" es mostra al catàleg, "venut" manté l'anunci com a històric i per tant deixa de ser visible per a la resta d'usuaris i "baixa" que també queda ocultat del llistat públic però pot tornar a "actiu".

La taula imatge_anunci conté les fotos de l'anunci i inclou un ordre per determinar com es veuran les imatges, fent que la primera fotografia actuï com a imatge principal.

Les disciplines constitueixen les 14 disciplines reconegudes per la Real Federación Hípica Española (RFHE) (salt, doma, etc.) relacionan't-se amb els anuncis mitjançant `anunci_disciplina`. Aquesta taula permet saber les disciplines de l'anunci evitant duplicats sobre la parella (anunci, disciplina).

Finalment, `revisió_anunci` permet que cada aprovació o rebuig quedi registrat amb l'usuari (administrador) que l'ha fet i el motiu.

5. IMPLEMENTACIÓ

Aquest capítol d'implementació s'organitza en set parts. (1) Tecnologies i eines: es justifica l'elecció de (HTML/CSS/JS per al frontend, Node.js + Express per al backend i MariaDB coma base de dades relacional) i les eines de suport (VSCode, Trello, Balsamiq, GitHub, etc.). (2) Configuració de l'entorn de desenvolupament: es detalla la instal·lació i posada a punt (Ubuntu, nvm/Node, MariaDB, DBeaver, variables d'entorn). (3) Codi: s'explica l'estructura MVC, l'organització de carpetes i els components clau de frontend i backend. (4) Codi open source: es documenta la llicència escollida i l'accés al repositori. (5) API: es descriu la capa REST i la seva documentació amb OpenAPI/Swagger. (6) Manual d'usuari: es una guia de l'ús de la plataforma (registrar-se, iniciar sessió, cercar/filtrar, publicar i administrar anuncis). (7) Instruccions de desplegament: es proporcionen els passos per restaurar la base de dades, configurar .env i arrencar el backend i frontend.

5.1. TECNOLOGIES

En aquest apartat es volen esmentar les diverses tecnologies que s'han escollit per a poder realitzar el treball. S'han inclòs 2 seccions, la primera amb les eines que no són tècniques i que han permès elaborar la memòria i mantenir el contacte amb els directors i la segona és on es poden trobar les diverses tecnologies utilitzades per a la creació i desenvolupament de la pàgina web.

5.1.1. Eines

Les eines utilitzades per a la gestió i el desenvolupament del projecte són les següents:

- **Google Docs:** document en línia utilitzat per a la redacció de la memòria i l'article, que permet la col·laboració dels directors, permetent que aquests facin correccions o comentaris pertinents.
- **Google Drive:** repositori compartit amb els directors que conté la documentació per al TFG (normativa, plantilles, criteris d'avaluació, memòries d'altres projectes, ...).
- **Google Sheets:** els ulls de càlcul han estat utilitzats per a planificar les hores necessàries per a cada tasca.
- **Google Meet:** s'ha utilitzat per a mantenir reunions bisetmanals amb els directors que han facilitat la col·laboració i acompanyament del projecte.
- **Gmail:** ha facilitat una comunicació efectiva amb els directors per a resoldre dubtes o problemes que han sorgit.
- **Balsamiq:** aquesta pàgina web ha permès dissenyar les interfícies d'usuari d'una manera ràpida i clara.
- **Trello:** m'ha permès organitzar les tasques de manera visual i fer-ne el seu seguiment, segons el mètode Kanban.

- **Visual Studio Code (VSCode):** conté una gran varietat d'extensions per al desenvolupament en JavaScript, Express.js, Git...
- **Github:** útil per emmagatzemar repositoris de codi font, amb funcions de control de versions i possibles col·laboracions per part dels implicats.
- **Chat GPT:** l'he utilitzat per a resoldre incidències de codi, i dur a terme tasques tedioses i repetitives, com l'elaboració del fitxer openapi.yaml.

5.1.2. Tecnologies de desenvolupament

Al frontal he utilitzat HTML5, CSS3 i JavaScript (ES6) per construir les pàgines i la interacció al navegador. Les crides al servidor es fan amb la Fetch API sobre HTTP. L'estil s'ha resolt amb CSS propi (sense frameworks de components), mantenint una estructura clara i classes reutilitzables.

Per al servidor he escollit Node.js i Express.js per construir els endpoints REST i servir contingut estàtic. La configuració CORS es fa amb el middleware cors per permetre el desenvolupament en ports separats (per exemple, 3001 backend i 5173 frontend). La gestió de variables sensibles (credencials de BBDD, ports, etc.) es fa amb dotenv. Per a la pujada d'imatges s'utilitza multer. La documentació d'API està disponible amb swagger-ui-express i l'arxiu openapi.yaml.

La base de dades és MariaDB, fent la connexió des del backend amb mysql2/promise per poder escriure codi net i eficient.

Frontend:

- **JavaScript:** Llenguatge de programació orientat a objectes utilitzat per crear funcionalitats interactives.
- **HTML:** Llenguatge que defineix l'estructura i el contingut de les pàgines web.
- **CSS:** Fulls d'estil en cascada per donar format i disseny visual a les pàgines web.

Backend:

- **Node.js:** Entorn d'execució per construir aplicacions backend.
- **Express.js:** Framework web minimalista per a Node.js que facilita la creació de servidors i APIs RESTful.
 - **Cors:** middleware d'Express per configurar i permetre peticions entre orígens diferents (Cross-Origin Resource Sharing).
 - **Multer:** middleware d'Express per gestionar pujades de fitxers, en aquest projecte s'ha utilitzat per a pujar les imatges dels anuncis

- **Dotenv:** llibreria que carrega variables d'entorn des d'un fitxer .env perquè el codi no exposi credencials ni configuració sensible.
- **Swagger:** conjunt d'eines (OpenAPI) per documentar i provar APIs.
- **Nodemon:** eina de desenvolupament que reinicia automàticament el servidor Node.js quan detecta canvis als fitxers.

Base de Dades:

- **MariaDB:** sistema de gestió de bases de dades relacionals, ràpid i escalable, amb codi obert i millores respecte a MySQL.
- **DBeaver:** aplicació per visualitzar i inspeccionar les taules de la base de dades durant el desenvolupament.

5.2. CONFIGURACIÓ DE L'ENTORN DE DESENVOLUPAMENT

A continuació es descriu com s'ha fet la instal·lació i quina configuració exacta ha estat utilitzada al projecte en un entorn d'Ubuntu.

1. Instal·lar eines i tecnologies bàsiques.

Primer de tot s'ha instal·lat Node.js i npm. Per fer una instal·lació correcta, cal utilitzar aquestes comandes a la terminal:

```
None
# Descarregar i instal·lar nvm:
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh |
bash
# Per reiniciar el shell
\ . "$HOME/.nvm/nvm.sh"
# Descarrega i instal·la Node.js:
nvm install 22
# Verifica la versió de Node.js:
node -v # Should print "v22.20.0".
# Verifica la versió de npm:
npm -v # Should print "10.9.3".
```

Codi 5.1. Instal·lació de Node.js i npm. Font: nodejs [\[11\]](#)

2. Configuració de la base de dades.

Per instal·lar MariaDB, s'han d'escriure aquestes instruccions a la terminal:

```
None
# Actualitzar la llista de paquets
sudo apt update
# Instal·lar el servidor de MariaDB
sudo apt install mariadb-server mariadb-client
# Assegurar la instal·lació
sudo mariadb-secure-installation
# Arrencar el servei de MariaDB
sudo systemctl start mariadb
sudo systemctl enable mariadb
# Verificar la versió instalada
mariadb -u root -p
```

Codi 5.2. Instal·lació i posada en marxa de MariaDB. Font: mariadb [\[12\]](#)

Per instal·lar DBeaver, s'han d'escriure aquestes instruccions a la terminal:

```
None
# Actualitzar la llista de paquets
sudo apt-get update
# Instal·lació de dbeaver
sudo snap install dbeaver-ce
```

Codi 5.3. Instal·lació de dbeaver. Font: dbeaver [\[10\]](#)

3. Configuració del backend

Es crea el directori /TFG/backend dins la carpeta /TFG. Un cop dins la carpeta, es genera el package.json.

```
None
cd /TFG/backend
#Generació del package.json
npm init -y
```

Codi 5.4. Inicialització del projecte Node.js per al backend. Font: npm documentation [\[13\]](#)

Per a instal·lar les altres biblioteques (express, cors, dotenv, mysql2, nodemon, multer i swagger), seguint a la carpeta del backend s'han d'executar aquestes comandes:

```
None
npm install express cors dotenv mysql2 multer swagger-ui-express ymljs

# Per a instal·lar nodemon, s'utilitza el paràmetre -D perquè s'instal·la
# com a dependència de desenvolupament
npm -D nodemon
```

Codi 5.5. Instal·lació de dependències del backend. Font: npm documentation [\[13\]](#)

Un cop ja s'han instal·lat llibreries i dependències, s'hauran de crear els directoris i fitxers per a que el backend quedi així:

```
None
servei-web/
└─ backend/
    ├── app.js
    ├── .env
    ├── package.json
    ├── config/
    │   └─ db.js
    ├── models/
    ├── routes/
    ├── controllers/
    ├── middlewares/
    ├── docs.js           (Swagger/OpenAPI)
    └─ openapi.yaml       (especificació)
```

Codi 5.6. Estructura dels directoris del backend. Font: elaboració pròpia

- /models: accés a dades (consultes SQL i mètodes del domini).
- /routes: arxius que mapegen URLs, agrupats per àrea (usuaris, anuncis, ...).
- /controllers: lògica de cada endpoint (crides a models, o respostes http).
- /middlewares: guarda les funcions reutilitzables (auth, multer, ...).

Fitxer app.js mínim funcional:

```
JSON
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```

Codi 5.7. Configuració mínima d'Express per arrencar el servidor. Font: express.js [\[14\]](#)

En el fitxer .env hi aniran les credencials per a la connexió amb la base de dades:

```
None
DB_HOST=localhost
DB_PORT=3306
DB_USER=usuari
DB_PASS=contrasenya
DB_NAME=nom_DB
APP_PORT=3001
```

Codi 5.8. Variables d'entorn per a la connexió a la base de dades. Font: npm dotenv [\[15\]](#)

Per a que la connexió a la base de dades funcioni, també s'ha d'omplir el fitxer db.js amb la informació de la connexió que hi ha al .env:

```
JavaScript
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10
});

module.exports = pool;
```

Codi 5.9. Pool de connexions MariaDB amb mysql2/promise. Font: stackoverflow [\[16\]](#)

Fitxer docs.js per publicar la documentació Swagger OpenAPI:

```
JavaScript
const path = require('path');
const swaggerUi = require('swagger-ui-express');
const YAML = require('yamljs');

module.exports = function mountDocs(app) {
  const openapiPath = path.join(__dirname, 'openapi.yaml');
  const spec = YAML.load(openapiPath);

  app.get('/openapi.json', (_req, res) => res.json(spec));
  app.use('/docs', swaggerUi.serve, swaggerUi.setup(spec, { explorer: true }));
};
```

Codi 5.10. Muntatge de documentació d'API amb Swagger UI. Font: elaboració pròpia.

4. Configuració frontend

Per a configurar el frontend, s'hauran de crear els següents directori i fitxers:

```
None
servei-web/
├─ backend/
└─ frontend/
    ├─ public/upload/anuncis/
    ├─ src/
    │   ├─ assets
    │   │   └─ styles.css
    │   ├─ components
    │   ├─ pages
    │   └─ scripts
    └-- index.html
```

Codi 5.11. Estructura de directoris del frontend. Font: elaboració pròpia

- /public/upload/anuncis: guarda les imatges pujades dels anuncis.
- /assets: recursos estàtics o compartits (css, icones, ...).
- /components: guarda les components d'UI reutilitzables (header, footer, ...).
- /pages: conté les diferents pantalles del projecte (els fitxers .html).
- /scripts: guarda els fitxers amb la lògica del client (els fitxers .js).

Per arrencar el servidor estàtic del frontend cal executar:

```
None  
cd /frontend  
npm run serve
```

Codi 5.12. Estructura de directoris del frontend. Font: docs.npmjs [\[17\]](#)

5. Integració amb github

Per fer la integració del projecte amb el github cal iniciar un repositori local dins la carpeta principal /TFG:

```
None  
git init  
git add .  
git commit -m "Inici del projecte"  
# Caldrà substituir <usuari> per l'usuari de github  
git remote add origin https://github.com/<usuari>/TFG.git  
git push -u origin main
```

Codi 5.13. Inicialització, publicació i configuració del remot del repositori a GitHub. Font: GitHub documentation [\[18\]](#)

Seguint pas a pas aquestes instruccions, quedarà creat i configurat l'entorn de desenvolupament del projecte

5.3. CODI

En aquest apartat es mostra com s'ha implementat la lògica del projecte tant al backend com al frontend, prioritzant la simplicitat i la separació de responsabilitats. Al servidor s'ha adoptat el patró MVC (Model View Controller) amb Express + MySQL per encapsular l'accés a dades, la validació d'entrades i la publicació d'API REST.

Per entendre com s'ha aplicat el patró, s'explica el flux complet que es produeix quan l'usuari vol veure els anuncis: La *Vista* (frontend) fa una petició HTTP (fetch) a l'API, Express deriva la petició a una ruta, que crida un *Controlador*, el qual delega l'accés a dades al *Model*.

Exemple d'implementació del patró MVC:

1. Vista

L'usuari accedeix a l'apartat d'anuncis, o apreta el botó per Aplicar els filtres. El client construeix la query i fa la crida a l'endpoint d'anuncis a través.

JavaScript

```
fetch(`http://localhost:3001/anuncis${qs ? `?${qs}` : ''}`);
```

Codi 5.14. Crida del client a l'endpoint d'anuncis amb els filtres seleccionats. Font: elaboració pròpia.

2. Enrutament

Al servidor, el fitxer app.js s'encarrega de muntar les rutes. En aquest cas qualsevol petició a /anuncis es delega a rutes_anuncis.js. Aquí només s'organitzen les URL i es vinculen als seus mòduls.

JavaScript

```
const anuncisRuta = require('./rutes/rutes_anuncis');  
app.use('/anuncis', anuncisRuta);
```

Codi 5.15. Aquest fragment de codi és el responsable de delegar les peticions que es fan a través del fetch. Font: elaboració pròpia.

3. Ruta específica

Al fitxer rutes_anuncis.js es defineix la ruta concreta i s'assigna la petició http al controlador que correspon:

JavaScript

```
const express = require('express');  
const router = express.Router();  
const ctrl = require('../controllers/anunci_controller');  
router.get('/', ctrl.llistar);
```

Codi 5.16. Definició de la ruta GET "/" d'anuncis i associació al controlador llistar. Font: elaboració pròpia.

4. Controlador

El fitxer `anunci_controller.js` rep la petició, llegeix els paràmetres (filtres, paginació), aplica les polítiques d'entrada (valors per defecte, tipus), i delega la consulta a un component especialitzat en els filtres.

```
JavaScript
const pool    = require('../config/db');
const model   = require('../models/anunci_model');
const filtres = require('../middlewares/filtres_anuncis');
// Importem NOMÉS el que necessitem del model de revisions
const { afegir: afegirRevisio } =
require('../models/revisio_anunci_model');

// Llistat públic (o amb filtres via query). Gestionat al middleware
async function llistar(req, res) {
  try {
    const rows = await filtres.llistarAmbFiltres(pool, req.query);
    res.json(rows);
  } catch (e) {
    console.error('[anuncis] Error llistant amb filtres:', e);
    res.status(500).json({ error: 'Error llistant anuncis' });
  }
}
```

Codi 5.17. Controlador llistar: s'encarrega de la lògica, delega els filtres i retorna la resposta. Font: elaboració pròpia.

5. Middlewares

Per separar responsabilitats, el mòdul `filtres_anuncis.js` construeix la consulta segons els paràmetres rebuts. La funció `llistarAmbFiltres` construeix dinàmicament l'SQL i els paràmetres (placeholder ?) en funció dels filtres, després executa la consulta.

```
JavaScript
async function llistarAmbFiltres(pool, q) {
  const { sql, params } = buildListQuery(q);
  const [rows] = await pool.query(sql, params);
  return postProcess(rows);
}
module.exports = {buildListQuery, llistarAmbFiltres};
```

Codi 5.18. Construeix SQL + paràmetres a partir de la query i retorna el resultat postprocessat. Font: elaboració pròpia.

6. Model

Per últim, `anuncisModel.js` s'encarrega d'encapsular l'accés a la base de dades i normalitzar-ne les dades obtingudes. I en funció dels filtres, aplica la paginació i retorna les dades obtingudes.

```
JavaScript
const pool = require('../config/db');
function normalize(a) {
  return {
    ...a,
    destacat: a.destacat === 1 || a.destacat === '1' || a.destacat ===
true,
    preu: a.preu !== null ? parseFloat(a.preu) : null,
    lat: a.lat !== null ? parseFloat(a.lat) : null,
    lon: a.lon !== null ? parseFloat(a.lon) : null
  };
}

async function llistar() {
  const [rows] = await pool.query(`
    SELECT anunci_id, usuari_id, nom, raca, preu, data_naixement,
      capa, alcada, pes, sexe, lat, lon, destacat, estat,
disponibilitat, descripcio,
      creat_el, actualitzat_el
    FROM anuncis
    ORDER BY creat_el DESC
  `);

  // adjuntem disciplines a cada anunci
  for (const a of rows) {
    const [d] = await pool.query(`
      SELECT d.nom FROM anunci_disciplina ad JOIN disciplines d ON
d.disciplina_id = ad.disciplina_id WHERE ad.anunci_id = ?
    `, [a.anunci_id]);
    a.disciplines = d.map(x => ({ nom: x.nom }));
  }
  return rows.map(normalize);
}
module.exports = {llistar}
```

Codi 5.19. Consulta base, enriquiment amb disciplines i normalització de tipus. Font: elaboració pròpia.

7. Resposta i renderització al client

Amb el flux MVC complet, el controlador retorna la resposta. El client rep les dades, actualitza la graella d'anuncis i el mapa i sincronitza la barra lateral de filtres. Quan l'usuari canvia filtres, el cicle es repeteix, seguint el mateix patró MVC però amb noves dades.

Com a funcionalitats destacades, s'explica el mapa interactiu (Mapbox GL JS) incloent l'ús del token, la configuració de l'estil i la col·locació de marcadors.

Funcionament dels marcadors en el mapa:

En aquest subapartat s'explica com es creen i es mostren els marcadors dels anuncis al mapa amb Mapbox GL JS. A partir de les dades retornades per l'API (lat, lon, nom, preu), el client genera una icona per a cada anunci, la posiciona a les coordenades corresponents i hi associa un popup informatiu.

Per a poder utilitzar un mapa interactiu amb tantes funcionalitats com el de google maps però que permet l'ús d'APIs de manera gratuïta, he decidit registrar-me a Mapbox GL JS i obtenir una llicència (access token). Aquest token autoritza l'ús de la API de Mapbox. En el desenvolupament el vaig col·locar al codi per simplicitat, tot i que en un entorn de producció és preferible injectar-lo via variables d'entorn per evitar la seva exposició pública.

1. Instància del mapa

Per a crear el mapa s'ha definit el contenidor HTML on s'insereix el mapa, un estil bàsic per a la seva claredat, i uns paràmetres inicials de vista (les coordenades de Vilanova i la Geltrú per a que el mapa carregui prop de la facultat). Aquesta configuració permet que l'usuari vegi immediatament l'àrea d'interès sense haver d'interactuar.

```
JavaScript
const map = new mapboxgl.Map({
  container: 'map',
  style: 'mapbox://styles/mapbox/streets-v12',
  center: [2.24741, 41.45004],
  zoom: 13.5
});
```

Codi 5.20. Creació i configuració inicial del mapa. Font: elaboració pròpia.

2. Fetch dels anuncis i col·locació de markers

Quan el mapa està llest, s'obtenen els anuncis de la API. El backend retorna un JSON amb els següents camps de l'anunci: lat i lon, nom i preu.

Es crea una icona (img) que fa de marcador de l'anunci, posicionant-se a les coordenades [a.lon, a.lat], s'afegeix un popup amb el nom i preu i per últim s'afegeix al mapa i es crida `updateSidebar()` per a sincronitzar el mapa i la llista.

D'aquesta manera es poden separar les responsabilitats: el servidor filtra i ordena i el client pinta el resultat.

```
JavaScript
fetch('http://localhost:3001/anuncis')
  .then(res => res.json())
  .then(anuncis => {
    allAnuncis = anuncis;

    anuncis.forEach(a => {
      const cavall = document.createElement('img');
      cavall.src = '/src/assets/horse-marker1.png'; // assegura ruta
pública
      cavall.alt = a.nom;
      cavall.style.width = '32px';
      cavall.style.height = '32px';
      cavall.style.cursor = 'pointer';

      new mapboxgl.Marker({ element: cavall, anchor: 'bottom' })
        .setLngLat([a.lon, a.lat])
        .setPopup(
          new mapboxgl.Popup({ offset: 25 })

        .setHTML(`<strong>${a.nom}</strong><br>${Number(a.preu).toFixed(2)} €`)
        )
        .addTo(map);
    });

    updateSidebar(); // render inicial
  })
  .catch(err => console.error('Error carregant anuncis al mapa', err));
```

Codi 5.21. Recuperació d'anuncis i renderització de markers i popups al mapa. Font: elaboració pròpia.

5.4. CODI OPEN SOURCE

Per aquest projecte s'ha escollit la llicència Open Source Apache 2.0 (<https://opensource.org/license/apache-2-0>). El codi del projecte està disponible al repositori de codi obert GitHub (<https://github.com/Cesconcins/tfg.git>).

Escollint aquesta llicència, cada col·laborador (jo) dóna permís de manera permanent, a tothom i sense exclusivitat, perquè qualsevol persona pugui utilitzar gratuïtament l'obra: usar-la, copiar-la, modificar-la per crear obres derivades, mostrar-la o executar-la en públic, concedir-ne subllicències i distribuir tant l'obra original com les versions modificades, ja sigui en forma de codi font o de codi compilat.

5.5. DOCUMENTACIÓ DE LA API

La plataforma fa servir una API REST com a capa de servei entre el frontend i el backend, seguint un estil RESTful i documentada amb OpenAPI. Les dades es comuniquen principalment en format JSON, amb documentació interactiva a /docs. El disseny dels endpoints segueix el model de recursos del domini (Usuaris, Perfil, Anuncis, Imatges,

Disciplines i un paquet d'Administració), de manera que les rutes reflecteixen clarament quina és l'operació de cada recurs: GET per consultar, POST per crear, PUT per modificar, PATCH per a canvis parcials i DELETE per eliminar.

Totes les rutes estan descrites a Swagger amb el propòsit, paràmetres i codis de resposta, la qual cosa facilita entendre, provar i integrar les operacions CRUD i d'administració.

Per a consultar la documentació, amb el backend en servei, s'haurà de navegar a la direcció *http://localhost:3001/docs*.

Usuaris Autenticació i gestió d'usuaris			^
GET	/usuaris/whoami	Retorna l'usuari autenticat	🔒 ↓
PUT	/usuaris/whoami	Actualitza el perfil de l'usuari autenticat	🔒 ↓
DELETE	/usuaris/whoami	Elimina el compte de l'usuari autenticat i fa logout	🔒 ↓
POST	/usuaris	Registre d'un nou usuari	↓
POST	/usuaris/login	Login d'usuari amb cookie de sessió	↓
POST	/usuaris/logout	Logout (expira cookie 'sid')	↓
GET	/usuaris/{id}	Obté un usuari per id	↓
DELETE	/usuaris/{id}	Elimina un usuari per id (ADMIN)	🔒 ↓

Figura 5.1. Endpoints dels usuaris. Font: elaboració pròpia amb OpenAPI [\[19\]](#).

Plataforma de Comerç Online per a la Venda de Cavalls

Francesco Oncins Spedo

The screenshot displays an OpenAPI interactive interface for the endpoint `/usuaris` with the operation `Registre d'un nou usuari`. The interface is divided into two main sections: **Parameters** and **Responses**.

Parameters Section:

- Method:** POST
- Path:** /usuaris
- Operation:** Registre d'un nou usuari
- Parameters:** No parameters
- Request body:** required, type `application/json`
- Example Value:**

```
{  "administrador": false,  "nom": "string",  "cognom": "string",  "telefon": "string",  "correu_electronic": "user@example.com",  "contraSenya": "string",  "actiu": true}
```

Responses Section:

Code	Description	Links
201	Creat Media type: <code>application/json</code> Controls Accept header. Example Value Schema <pre>{ "usuari_id": 0, "nom": "string", "cognom": "string", "correu_electronic": "user@example.com", "telefon": "string", "administrador": true}</pre>	No links
400	Camps invàlids o duplicats Media type: <code>application/json</code> Example Value Schema <pre>{ "error": "string"}</pre>	No links

Figura 5.2. Endpoint de l'operació registre d'un usuari. Font: elaboració pròpia amb OpenAPI [\[19\]](#).

La **Figura 5.2.** mostra l'estructura estandaritzada que segueixen els endpoints de la plataforma, amb la definició dels paràmetres d'entrada i de les respostes esperades. En l'exemple del registre, s'indiquen els camps requerits i les possibles sortides, 201 (Created) en cas d'alta correcta i 400 (Bad Request) per a validacions fallides o correu duplicat. La documentació interactiva generada amb OpenAPI, facilita la verificació i prova dels serveis en temps real, permeten executar crides des del mateix swagger, i obtenint una resposta immediata i clara.

Plataforma de Comerç Online per a la Venda de Cavalls

Francesco Oncins Spedo

Perfil Operacions sobre el perfil autenticat			^
GET	/perfil	Retorna el perfil autenticat	🔒 ↓
PUT	/perfil	Actualitza dades del perfil	🔒 ↓
DELETE	/perfil	Esborra el compte i tanca la sessió	🔒 ↓
GET	/perfil/anuncis	Llista anuncis de l'usuari autenticat	🔒 ↓
Anuncis CRUD d'anuncis			^
GET	/anuncis	Llista anuncis (públic) amb filtres opcionals	↓
POST	/anuncis	Crea un anunci (cal login)	🔒 ↓
GET	/anuncis/{id}	Detall d'un anunci	↓
PUT	/anuncis/{id}	Actualitza un anunci (propietari o admin)	🔒 ↓
DELETE	/anuncis/{id}	Esborra un anunci (propietari o admin)	🔒 ↓
PATCH	/anuncis/{id}/estat	Canvia l'estat d'un anunci (ADMIN)	🔒 ↓
GET	/anuncis/{id}/revisions	Llista l'històric de revisions d'un anunci (ADMIN)	🔒 ↓
Imatges Gestió d'imatges d'anuncis			^
GET	/anuncis/{id}/imatges	Llista imatges d'un anunci	↓
POST	/anuncis/{id}/imatges	Puja imatges per a un anunci (propietari o admin)	🔒 ↓
PATCH	/anuncis/{id}/imatges/ordre	Reordena les imatges d'un anunci	🔒 ↓
GET	/anuncis/{id}/portada	Redirigeix a la portada (primera imatge) d'un anunci	↓
DELETE	/anuncis/imatges/{imgId}	Esborra una imatge concreta d'un anunci	🔒 ↓
Disciplines Llistat de disciplines			^
GET	/disciplines	Llista totes les disciplines	↓
Admin Endpoints d'administració			^
GET	/admin/dashboard	Mètriques bàsiques del panell d'administració	🔒 ↓

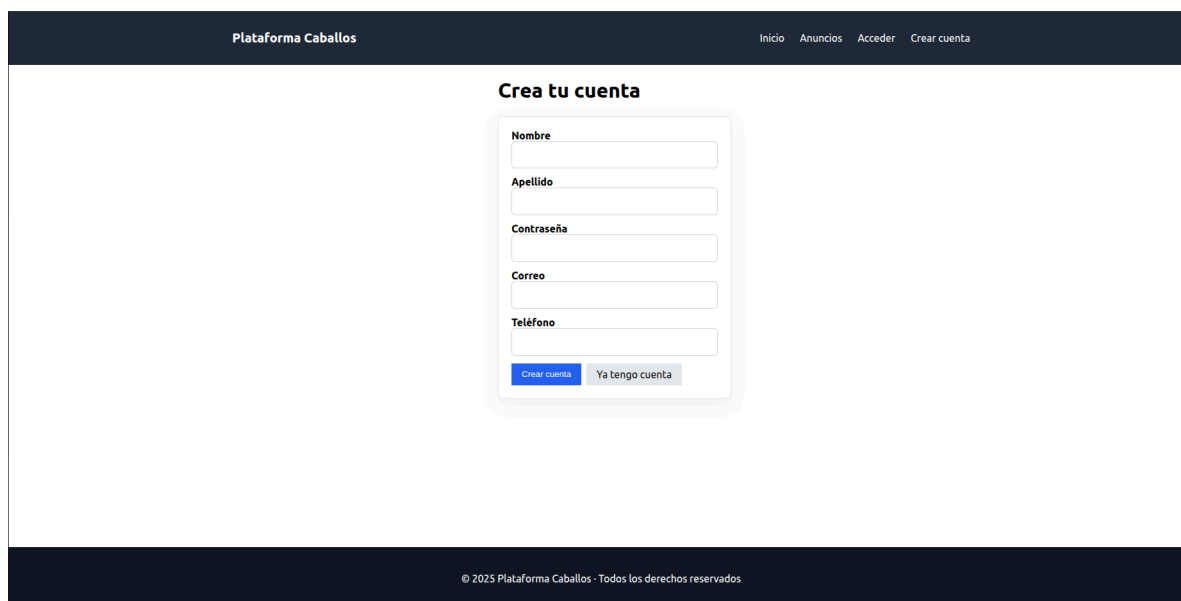
Figura 5.3. Endpoints dels recursos restants del codi. Font: elaboració pròpia amb OpenAPI [\[19\]](#).

5.6. MANUAL DE L'USUARI

En aquest apartat de la memòria es pretén explicar com usar la pagina web de manera correcta, amb instruccions pas a pas per a registrar-se, accedir, cercar i filtrar anuncis, consultar-ne els detalls, publicar-ne de nous i administrar-ne el contingut.

Registrar l'usuari

Per publicar anuncis o gestionar el perfil, cal registrar-se.



La imatge mostra la interfície web de la Plataforma Caballos per a la creació d'un compte. A la part superior, hi ha una barra de navegació amb el nom de la plataforma i enllaços a Inicio, Anuncios, Acceder i Crear cuenta. El títol principal de la secció és "Crea tu cuenta". A sota, hi ha un formulari amb camps per a Nombre, Apellido, Contraseña, Correo i Teléfono. Al final del formulari, hi ha dos botons: "Crear cuenta" (en blau) i "Ya tengo cuenta" (en gris). A la part inferior de la pàgina, hi ha un peu de pàgina que indica "© 2025 Plataforma Caballos - Todos los derechos reservados".

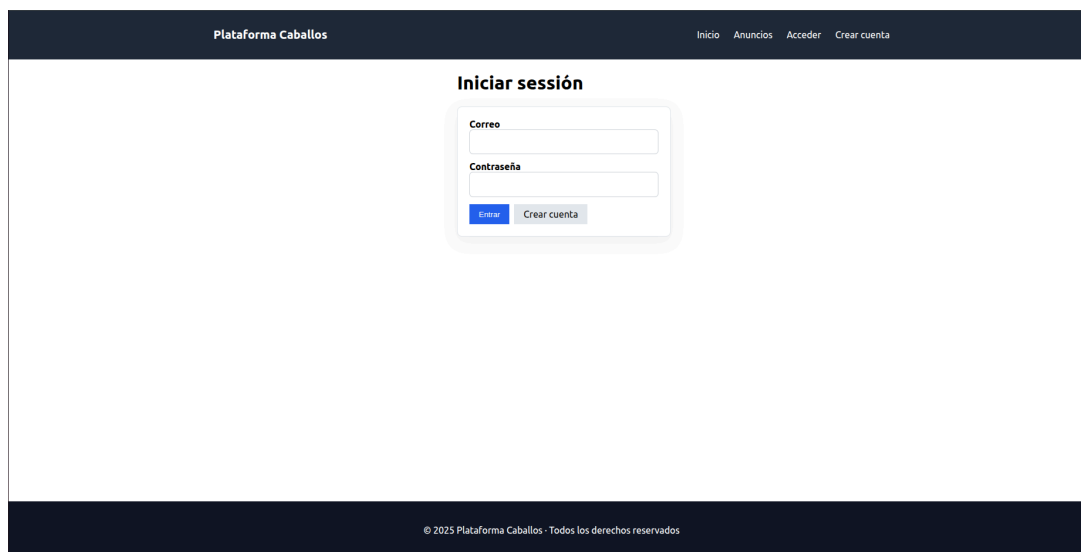
Figura 5.4. Captura de pantalla de la creació d'un compte. Font: elaboració pròpia

Passos:

1. Omple els camps obligatoris.
2. Prem Crear compte. El sistema identificarà si l'usuari ja existeix.
3. Si ja tens compte, pots anar a l'accés amb el botó "Ya tengo cuenta"

Iniciar sessió

Per a poder iniciar sessió a plataforma s'ha d'introduir el correu electrònic i contrasenya del compte que s'ha creat prèviament.



Plataforma Caballos

Inicio Anuncios Acceder Crear cuenta

Iniciar sesión

Correo

Contraseña

[Entrar](#) [Crear cuenta](#)

© 2025 Plataforma Caballos - Todos los derechos reservados

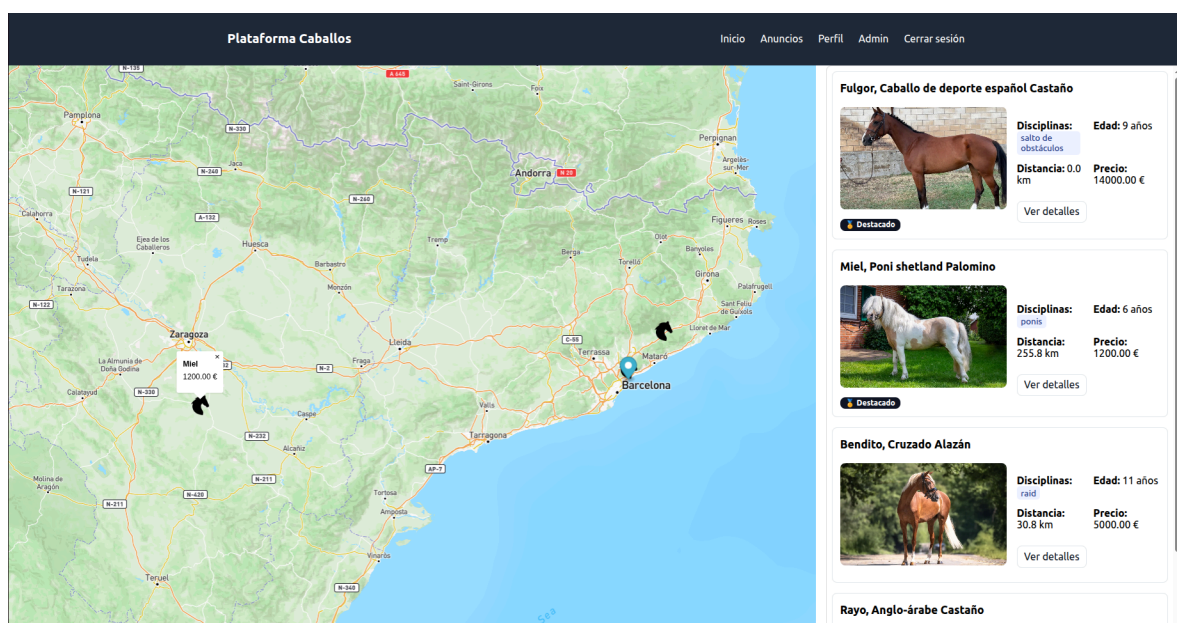
Figura 5.5. Captura de pantalla de l'inici de sessió. Font: elaboració pròpia

Passos:

1. Introdueix el correu i la contrasenya
2. Si no tens compte prem el botó “Crear Cuenta” per registrar-te

Pàgina d'inici

A la pàgina inicial es mostra un mapa interactiu amb els anuncis de cavalls i, al costat, un llistat en targetes. Cada targeta presenta el títol amb “Nom, raça capa”, la foto principal, les disciplines, l'edat, la distància (si n'hi ha) i el preu. Els anuncis destacats apareixen ressaltats.



Plataforma Caballos

Inicio Anuncios Perfil Admin Cerrar sesión

Fulgor, Caballo de deporte español Castaño

Disciplinas: salto de obstáculos **Edad:** 9 años

Distancia: 0.0 km **Precio:** 14000.00 €

Destacado [Ver detalles](#)

Miel, Poni shetland Palomino

Disciplinas: ponis **Edad:** 6 años

Distancia: 255.8 km **Precio:** 1200.00 €

Destacado [Ver detalles](#)

Bendito, Cruzado Alazán

Disciplinas: raid **Edad:** 11 años

Distancia: 30.8 km **Precio:** 5000.00 €

Destacado [Ver detalles](#)

Rayo, Anglo-árabe Castaño

Figura 5.6. Captura de pantalla de la pàgina d'inici. Font: elaboració pròpia

Pel que fa a la interacció amb el mapa, l'usuari pot permetre la geolocalització perquè es mostri la seva posició i els anuncis propers. En tocar un marcador, s'obre un resum amb el nom i el preu, i pot moure i fer zoom al mapa per explorar diferents zones.

La llista lateral està ordenada mostrant primer els anuncis destacats i després per distància relativa a l'usuari, es pot revisar ràpidament les característiques bàsiques de cada cavall, identificar si és destacat, i obrir la fitxa completa amb el botó "Ver detalles" per veure tota la informació.

Anuncis

A la pàgina d'anuncis, l'usuari hi troba un catàleg clar i visual amb les ofertes actives. Cada anunci es presenta en forma de targeta amb la fotografia principal, el nom del cavall i la raça + capa com a informació més rellevant. Més avall s'hi mostren les disciplines associades al cavall, edat, preu i sexe. Un distintiu permet identificar els anuncis marcats com a destacats.

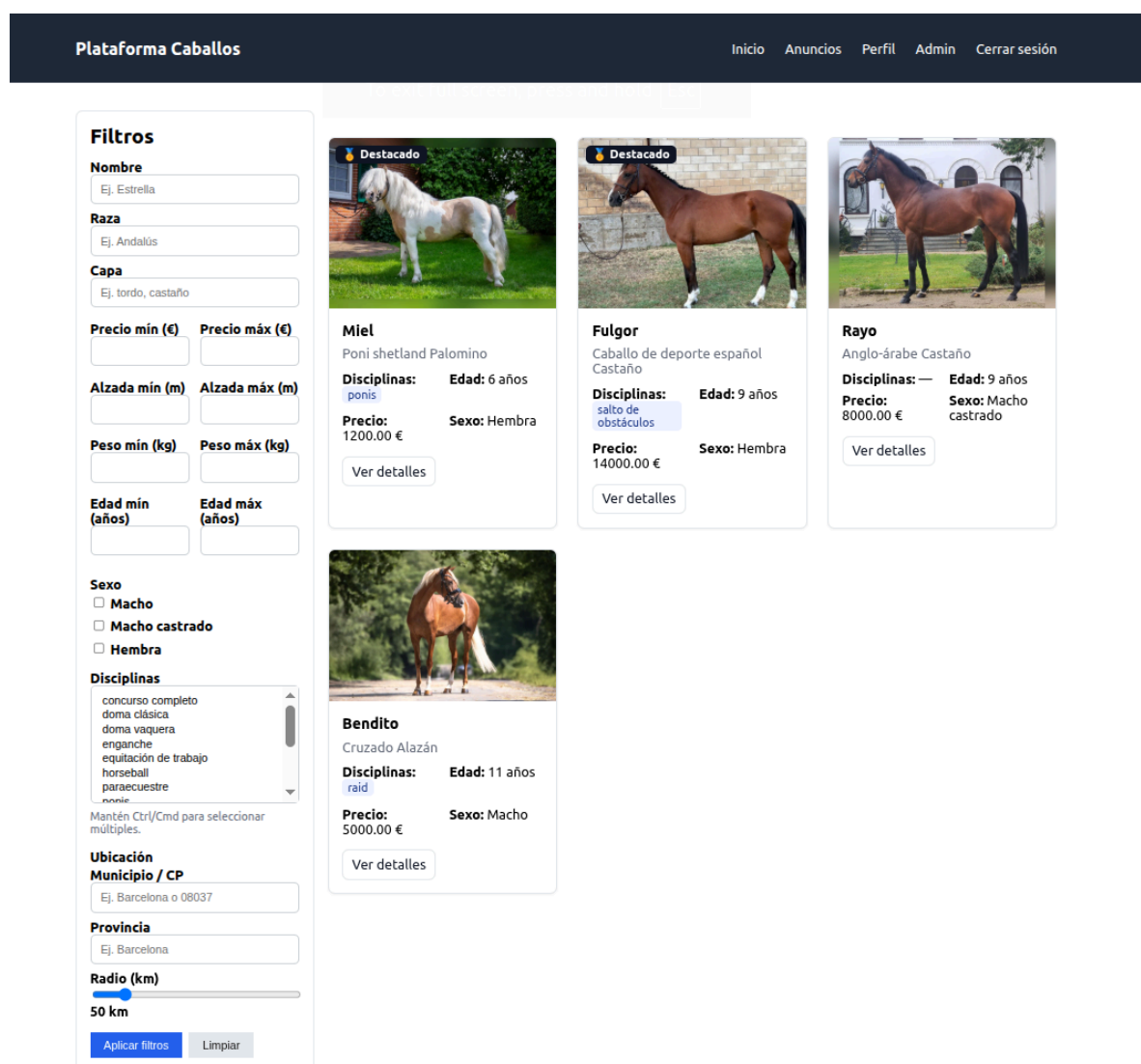


Figura 5.7. Captura de pantalla de la pàgina d'anuncis sense filtres. Font: elaboració pròpia

Pel que fa a les accions, l'usuari pot ajustar els filtres disponibles del lateral per afinar la cerca segons les seves preferències. D'aquesta manera, pot combinar criteris com la raça, les disciplines i el rang de preu. Un cop aplicats o modificats els filtres, s'haurà d'apretar el botó "Aplicar Filtros" per a què la graella d'anuncis s'actualitzi.

Finalment, des de cada targeta es pot accedir a la fitxa completa de l'anunci mitjançant el botó "Ver detalles". Aquesta acció porta a una vista amb tota la informació ampliada de l'animal. D'aquesta manera, l'usuari pot explorar el catàleg, filtrar el que necessita i aprofundir fàcilment en els anuncis que li cridin més l'atenció.

A continuació es mostren 2 captures de pantalla que mostren el funcionament dels filtres.

La imatge mostra la interfície d'usuari de la plataforma "Plataforma Caballos". A la part superior, hi ha una barra de navegació amb els enllaços: Inicio, Anuncios, Perfil, Admin i Cerrar sesión. A l'esquerra, hi ha un panel de filtres amb els següents camps:

- Filtros**
- Nombre**: Ej. Estrella
- Raza**: Ej. Andalus
- Capa**: Ej. tordo, castaño
- Precio mín (€)** i **Precio máx (€)**: Camps buits.
- Alzada mín (m)** i **Alzada máx (m)**: Camps buits.
- Peso mín (kg)** i **Peso máx (kg)**: Camps buits.
- Edad mín (años)** i **Edad máx (años)**: Camps buits.
- Sexo**:
 - ☐ Macho
 - ☐ Macho castrado
 - ☒ Hembra
- Disciplinas**:
 - ponis (seleccionat)
 - raid
 - reining
 - salto de obstáculos
 - trec
 - turismo ecuestre
 - volteo
- Mantén Ctrl/Cmd para seleccionar múltiples.
- Ubicación**:
 - Municipio / CP**: Ej. Barcelona o 08037
 - Provincia**: Ej. Barcelona
- Radio (km)**: Slider ajustat a 50 km.
- Botons: **Aplicar filtros** i **Limpiar**.

A la dreta, hi ha un anunci destacat amb el títol "Destacado" i una imatge d'un cavall. El text de l'anunci és:

- Miel**
- Poni shetland Palomino
- Disciplinas:** ponis
- Edad:** 6 años
- Precio:** 1200.00 €
- Sexo:** Hembra
- Botó: **Ver detalles**

Figura 5.8. Captura de pantalla de la pàgina d'anuncis amb filtres de disciplines i sexe. Font: elaboració pròpia

Plataforma Caballos

Inicio

Anuncios

Perfil

Admin

Cerrar sesión

Filtros

Nombre

Ej. Estrella

Raza

Ej. Andalus

Capa

Castaño

Precio mín (€)

Precio máx (€)

10000

Alzada mín (m)

Alzada máx (m)

Peso mín (kg)

Peso máx (kg)

Edad mín (años)

Edad máx (años)

Sexo

☐ Macho

☐ Macho castrado

☐ Hembra

Disciplinas

ponis

raid

reining

salto de obstáculos

trec

turismo ecuestre

volteo

Mantén Ctrl/Cmd para seleccionar múltiples.

Ubicación

Municipio / CP

Ej. Barcelona o 08037

Provincia

Ej. Barcelona

Radio (km)

50 km

Aplicar filtros

Limpiar



Rayo

Anglo-árabe Castaño

Disciplinas: —

Edad: 9 años

Precio: 8000.00 €

Sexo: Macho castrado

Ver detalles

Figura 5.9. Captura de pantalla de la pàgina d'anuncis amb filtres de Capa i preu màxim. Font: elaboració pròpia

55

Detalls de l'anunci

A la pàgina de detalls d'un anunci, l'usuari troba d'entrada una galeria d'imatges que permeten veure les diferents fotos del cavall que han estat penjades, el títol i la raça + capa. A la part superior del cos de la fitxa s'hi mostren les dades essencials disciplina, raça, edat, alçada i ubicació juntament amb un possibles distintius que permet saber si l'anunci és destacat o no. Just a sota hi ha la informació del venedor, nom, cognoms i correu_electrònic, i a la dreta es pot veure la descripció completa del cavall.



Bendito
Cruzado Alazán

Información principal			
Precio	5000.00 €	Sexo	Macho
Edad	11 años	Alzada	1.58 cm
Peso	500.00 kg	Disciplinas	raid

Descripción
Caballo súper dócil, perfecto para rutas y iniciación. Vive en paddock, fácil de manejar y herrar.

Contacto
francesco oncins spedo
francesco@gmail.com

Figura 5.10. Captura de pantalla de la pàgina detalls d'anunci. Font: elaboració pròpia

Perfil

La pàgina de perfil presenta un espai clarament separat en 2: a l'esquerra, una Àrea personal compacta amb les dades bàsiques (nom, cognoms, correu i telèfon), i si el compte té permisos, també apareix un enllaç cap al panell d'administració. A la dreta, la secció "Mis anuncios" mostra els anuncis del propi usuari en una graella amb imatge, preu i informació essencial.

Pel que fa a les accions sobre el compte, l'usuari pot editar les seves dades amb el botó "Editar". Com que el correu es l'identificador, no és editable, mentre que la resta de dades si que ho són. També es pot eliminar el compte amb el botó adient.

Pel que fa als anuncis, l'usuari pot crear-ne un apretant el botó "Crear anuncio", editar-ne qualsevol existent pitjant el botó "Editar", veure'n els detalls a través de l'enllaç "Ver

detalles”, marcar-lo com a venut, donar-lo de baixa o eliminar-lo de manera definitiva .

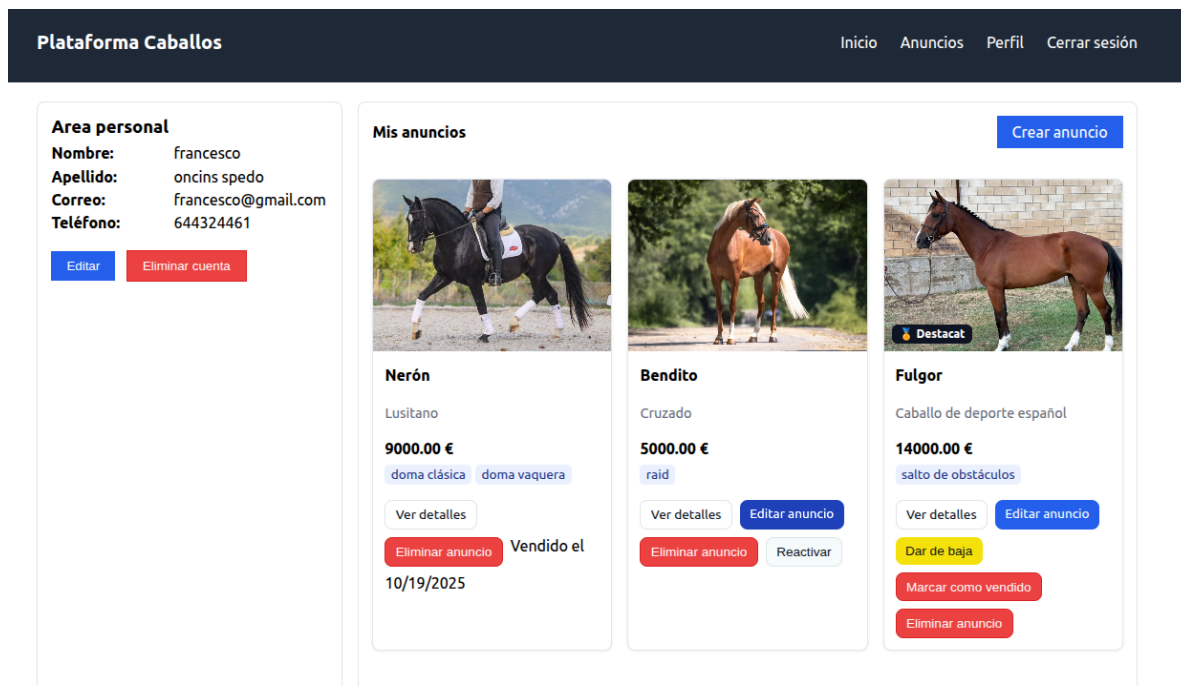


Figura 5.11. Captura de pantalla de la secció del perfil d'usuari. Font: elaboració pròpia

Crear anunci

La pàgina mostra un formulari amb les dades bàsiques per a la creació de l'anunci (títol, preu, disciplina, raça, edat i alçada), ubicació a través de codi postal, població i direcció o mapa interactiu, capa, alçada, pes, sexe (podent escollir entre "Macho", "Hembra" o "Macho Castrado", descripció i fotos.

Si s'hi accedeix amb un identificador d'anunci, el formulari es carrega amb les dades de l'anunci per editar-lo.

Plataforma Caballos

Inicio Anuncios Perfil Admin Log Out

Crear anuncio

Nombre del caballo

Raza

Precio (€)

Fecha de nacimiento
mm/dd/yyyy

Disciplinas
concurso completo
doma clásica
doma vaquera
enganche
equitación de trabajo
horseball
paraecuestre
none

Mantén Ctrl/Cmd per seleccionar múltiples.

Ubicación

Código postal

Población

Dirección

Capa

Alzada (m)

Peso (kg)

Sexo

Descripción

Imágenes
Selecciona imágenes (puedes elegir varias)
Choose Files No file chosen
Tamaño máximo por archivo: 5MB.

Guardar Cancelar

Figura 5.12. Captura de pantalla del formulari de creació d'anuncis. Font: elaboració pròpia

Pel que fa a les accions, l'usuari pot publicar o guardar canvis amb el botó corresponent (en blau), cancel·lar l'operació, ajustar la ubicació al mapa, pujar o eliminar fotografies i obrir la fitxa amb l'enllaç "Ver detalles" (compacte). Els errors de validació es mostren al costat del camp afectat.

Administrador

La pàgina presenta un contador d'anuncis i usuaris, amb filtres per als usuaris (activo, inactivo) i per als anuncis (estat, disciplina, data), el llistat d'usuaris i el llistat d'anuncis (pendents, aprovats o rebutjats). Tot es mostra en format compacte per revisar ràpidament el catàleg.

Pel que fa a les accions, l'administrador pot aprovar o rebutjar anuncis, obrir-ne la fitxa amb "Ver detalles", marcar-los com a "Destacado" i eliminar-los.

Plataforma de Comerç Online per a la Venda de Cavalls

Francesco Oncins Spedo

Plataforma Caballos

InicioAnunciosPerfilAdminCerrar sesión

Panel del administrador

Resumen

Usuarios: 4

Anuncios: 6

Usuarios

Estado: Todos

Jordi g	Activo	Editar	Activar	Eliminar
No Activos: 5 - Anuncios: 2				

Pau Alcázar Activo

pau@gmail.com

121212121 - Alta: 16/10/2025 - Anuncios: 0

EditarDesactivarEliminar

francesco oncins spedo Activo

francesco@gmail.com

644324401 - Alta: 11/9/2025 - Anuncios: 3

EditarDesactivarEliminar

Admin Admin Activo Admin

admin@root.com

999999999 - Alta: 22/5/2025 - Anuncios: 1

EditarDesactivarEliminar

Anuncios

Estado: Todos

	Rayo Pendiente	Ver detalles	Validar	Rechazar	Eliminar	Destacar
Pura sangre - mascle castrat						
salto de obstáculos						

Fulgor Validado Destacado

Caballo de deporte español - femella

salto de obstáculos

Ver detallesValidarRechazarEliminarQuitar destacado

Bendito Validado

Cruzado - mascle

raïd

Ver detallesValidarRechazarEliminarDestacar

Nerón Validado

Lusitano - mascle castrat

doma clàssica, doma vaquera

Ver detallesValidarRechazarEliminarDestacar

Miel Validado Destacado

Poni shetland - femella

ponis

Ver detallesValidarRechazarEliminarQuitar destacado

Rayo Rechazado

Anglo-àrabe - mascle castrat

Ver detallesValidarRechazarEliminarDestacar

Figura 5.13. Captura de pantalla de la pàgina de l'administrador. Font: elaboració pròpia

5.7. INSTRUCCIONS PER AL DESPLEGAMENT

A continuació es descriu el procés de desplegament en un entorn local (Ubuntu). L'objectiu és posar en marxa backend (API) i frontend (interfície) amb la base de dades restaurada a partir d'un dump. Per a tots els prerequisits ja explicats, es farà referència a les seccions anteriors.

1. Requisits previs

- Node.js + npm: per a la instal·lació amb nvm de node.js i npm vegeu (**Codi 5.1.**). Si la màquina ja té una versió compatible, no cal repetir-ho.
- MariaDB: per a la instal·lació i arrencada de MariaDB vegeu (**Codi 5.2.**).

2. Clonar el repositori

Clonar el repositori des de github a la màquina on es farà l'execució:

```
None  
git clone https://github.com/Cesconcins/tfg.git
```

Codi 5.22 Clonació del repositori. Font: GitHub [\[20\]](#).

3. Crear i restaurar la base de dades

Entrar a MariaDB amb un usuari amb privilegis i crear la base de dades i l'usuari d'aplicació.

```
SQL  
-- Inici de sessió (terminal):  
-- mysql -u root -p  
CREATE DATABASE IF NOT EXISTS cavalls_db;  
CREATE USER IF NOT EXISTS 'Jordi'@'localhost' IDENTIFIED BY  
'JordiGarciaUPC';  
GRANT ALL PRIVILEGES ON cavalls_db.* TO 'Jordi'@'localhost';  
FLUSH PRIVILEGES;
```

Codi 5.23. Creació de la base de dades i usuari d'aplicació. Font: elaboració pròpia.

Importar el dump, que es troba al github, de la base de dades al nou esquema:

```
None  
mysql -u Jordi -p -h localhost -D cavalls_db < cavalls_db_dump.sql
```

Codi 5.24. Restauració del dump cavalls_db_dump.sql. Font: DigitalOcean [\[21\]](#).

4. Backend

Per a poder connectar el projecte amb la base de dades són necessaris 2 fitxers:

- Fitxer .env: per a modificar el fitxer .env amb les credencials i paràmetres correctes vegeu (**Codi 5.8.**).
- Fitxer db.js: per a la seva configuració vegeu (**Codi 5.9.**).

Un cop configurats aquests 2 arxius, s'han d'instal·lar dependències i arrencar el servidor:

```
None  
cd backend  
npm install  
npm run dev
```

Codi 5.25. Instal·lació i arrencada del backend. Font: dosc.npmjs [\[17\]](#).

5. Frontend

Per a la configuració del frontend vegeu (**Codi 5.12.**).

Per accedir via navegador a la web, anar a la direcció: *http://localhost:5173*

6. VALIDACIÓ

6.1. PROVES DE REQUISITS FUNCIONALS

Aquest apartat recull el conjunt de proves d'acceptació definides per verificar que el sistema compleix els requisits funcionals especificats. Cada prova descriu unes precondicions, un procediment, un resultat esperat i l'impacte a la base de dades, garantint la traçabilitat entre requisits i evidències.

El pla cobreix la gestió d'usuaris, l'autenticació/autorització, el cicle de vida d'anuncis (alta, edició, venda/baixa), la visualització pública, destacats (rol administrador), la cerca amb filtres i la integració amb mapa. En total s'han definit 36 proves, que al tancament del projecte passen satisfactòriament en l'entorn de validació indicat.

A continuació es presenten totes les proves de requisits funcionals executades.

6.1.1. Gestió d'usuaris

Alta d'usuaris

- **Alta correcta**
Pre: cap. Passos: Formulari complet (email únic, contrasenya forta, consentiments) → Enviar. Esperat: 201/redirect, perfil creat, email verificació (si s'aplica). DB: nou usuari amb rol Usuari i estat "actiu".
- **Email duplicat / validacions**
Pre: usuari existent amb email=a@a.com. Passos: Registrar-se amb el mateix correu. Esperat: 409/422 amb missatge "email en ús", cap alta a DB.
- **Protecció contrasenya**
Pre: cap. Passos: Postman: alta i després GET /users (com a Usuari). Esperat: mai es retorni password/hash en respostes; a DB hi ha hash (no en clar).

Modificació de perfil

- **Edició bàsica**
Pre: usuari autenticat. Passos: Canviar nom, telèfon i preferències de notificació. Esperat: 200 i valors persistits; relectura del perfil mostra canvis.
- **Permisos**
Pre: 2 usuaris A i B. Passos: B intenta editar perfil d'A. Esperat: 403; cap canvi a DB.

Baixa d'usuaris

- **Sol·licitud de baixa**
Pre: usuari autenticat amb anuncis actius i preferits. Passos: Demanar baixa. Esperat: estat compte "eliminat/inhabilitat", logout forçat; anuncis passen a estat acordat (per

ex. “pausat” o “orfe” segons disseny), preferits es desvinculen.

- **Accés després de baixa**

Pre: compte en estat “eliminat/inhabilitat”. Passos: Intent login/refresh token. Esperat: 401; cap token nou.

Autenticació i autorització (rols)

- **Registre d'usuari correcte**

Pre: usuari no registrat. Passos: Register → accedir a endpoint protegit. Esperat: 200 i accés; expiració/refresh funciona.

- **Login correcte**

Pre: usuari registrat. Passos: Login → accedir a endpoint protegit. Esperat: 200 i accés; expiració/refresh funciona.

- **Accés per rol**

Pre: Usuari (rol=Usuari), Admin (rol=Administrador). Passos: Usuari prova endpoint d'admin (gestió destacats). Esperat: 403; Admin 200.

6.1.2. Gestió d'anuncis de venda de cavalls

Alta d'anuncis

- **Creació completa (amb estat)**

Pre: usuari autenticat. Passos: Formulari amb raça, edat, preu, ubicació, imatges, estat (“disponible/venut/reservat”), sexe i “femella castrada” quan pertoqui. Esperat: 201; llistat mostra targeta amb estat; a DB es guarden atributs i bandera “castrada”.

- **Validacions de camp**

Pre: cap. Passos: Preu negatiu. Esperat: 422 per cada cas; cap inserció.

- **Política d'imatges**

Pre: cap. Passos: Pujar fitxer no permès (p.ex. .exe). Esperat: 415/422;

Modificació d'anuncis

- **Editar propi anunci**

Pre: anunci A1 de l'usuari. Passos: Canviar preu i estat. Esperat: 200; historitza canvis (si hi ha auditories); al llistat es veu nou preu/estat.

- **Restringir edició d'altres**

Pre: usuari B, anunci d'A. Passos: B intenta editar A1. Esperat: 403; sense canvis.

Baixa d'anuncis / venda

- **Marcar com venut**

Pre: anunci en “disponible” amb comprador identificat. Passos: Propietari marca “venut” i selecciona comprador. Esperat: estat “venut”; només el comprador pot veure

el detall complet; resta d'usuaris veuen targeta "venut" sense detall.

- **Retirada sense venda**

Pre: anunci actiu. Passos: Donar de baixa. Esperat: no apareix en llistats públics; es conserva a DB per auditories.

Visualització d'anuncis (públic)

- **Usuari no autenticat**

Pre: catàleg amb diversos anuncis. Passos: Navegar llistat i entrar a un detall "disponible". Esperat: es veu informació bàsica i imatges; accions protegides (guardar, contactar) mostren CTA "inicia sessió".

- **Paginació**

Pre: +50 anuncis. Passos: Canviar de pàgina/limit. Esperat: es retorna llista paginada estable i ordenada segons criteri per defecte.

Anuncis preferits

- **Afegir/treure preferit**

Pre: usuari autenticat i anunci A1. Passos: Guardar com preferit i després treure. Esperat: 200; comptador/favorit reflectit al perfil.

- **Integritat**

Pre: usuari U i U2. Passos: U2 no veu els preferits d'U. Esperat: aïllament correcte.

6.1.3. Anuncis destacats (administrador)

- **Marcatge manual**

Pre: admin i 10 anuncis. Passos: Marcar 3 com a destacats. Esperat: 200; a la home apareixen al carrusel/secció "destacats" a dalt.

- **Desmarcar i ordre**

Pre: n'hi ha 3 destacats. Passos: Canviar l'ordre/posició i desmarcar-ne 1. Esperat: home reflecteix l'ordre; el desmarcat desapareix.

- **Permisos**

Pre: usuari normal. Passos: Cridar endpoint destacats. Esperat: 403.

6.1.4. Cerca i filtres

Filtres avançats (raça, edat, preu, ubicació, disponibilitat en esdeveniments)

- **Cerca multi-filtre vàlida**

Pre: índex amb dades variades. Passos: Filtrar per raça="Pura Sang", preu 3k–6k, edat 4–8, "disponible en esdeveniments=Si". Esperat: 200; només anuncis que compleixen tots els filtres; criteri d'ordenació: destacat i, si hi ha permís d'ubicació, distància.

- **Filtres invàlids**

Pre: cap. Passos: Enviar raça inexistent o rang preu invertit. Esperat: 422 amb missatge; cap consulta a DB o bé default safe.

- **Sense permís d'ubicació**

Pre: navegador sense permís/denegat. Passos: Aplicar filtres amb "ordenar per distància". Esperat: no es calcula la distància; s'ordena només per "destacat" (i criteri secundari establert: data, preu, etc.).

Proximitat i mapa

- **Cerca per radi**

Pre: ubicació usuari (lat/lon) i anuncis amb coordenades. Passos: Cercar dins 50 km. Esperat: només anuncis dins del radi; distància mostrada (si permès).

- **Anunci sense coordenades**

Pre: un anunci sense lat/lon. Passos: Llistat per distància. Esperat: l'anunci es mostra al final o exclòs segons regla; mai amb distància "0".

6.1.5. Mapa d'ubicacions

- **Render de pins i infoWindow**

Pre: 5 anuncis amb coordenades. Passos: Obrir mapa. Esperat: 5 pins; en clicar es veu resum i enllaç a detall.

- **Errors de geocodificació**

Pre: un anunci amb adreça no geocodificable. Passos: Obrir mapa. Esperat: es mostra amb estat "ubicació no disponible"; no trenca el mapa.

6.1.6. Panell de control de l'administrador

Gestió d'usuaris

- **Activar/desactivar**

Pre: usuari U. Passos: Admin desactiva U. Esperat: U no pot iniciar sessió (401) i queda marcat com "inactiu".

- **Edició de dades**

Pre: U. Passos: Admin edita nom/rol. Esperat: 200; nou rol aplicat immediatament (permisos).

Validació d'anuncis (flux d'estats)

- **Revisió i publicació**

Pre: anunci en estat "pendent". Passos: Admin canvia a "aprobat". Esperat: apareix en el catàleg públic.

- **Rebuig amb motiu**

Pre: anuncis pendents. Passos: Rebutjar amb missatge. Esperat: propietari veu el

motiu; pot editar i reenviar.

- **Llista de pendents/rebutjats**

Pre: varis estats. Passos: Consultar llistes. Esperat: paginades, filtrables; accions de “aprovar/rebutjar” funcionen.

6.2. ASSOLIMENT DELS REQUISITS NO FUNCIONALS

En aquest apartat es pretén demostrar els requeriments no funcionals que s’han pogut validar.

A diferència dels requisits funcionals, els requisits no funcionals rarament es verifiquen amb proves binàries; per això, l’assoliment s’acredita mitjançant evidències (organització modular del codi, comentaris, documentació i adaptabilitat visual mitjançant media queries).

Codi modular i documentat

El projecte segueix el patró MVC i organitza el codi per responsabilitats, fet que facilita el manteniment, la traçabilitat i l’evolució. El codi està separat en 2 carpetes, una per al backend i una altra per al frontend. Dins el backend podem trobar el directori de models (s’encapsulen les dades), el de controllers (contené els fitxers amb la lògica del projecte), el de rutes (on es defineixen els contractes http) i per últim el de middlewares (que guarda fitxers amb funcionalitats diverses).

Si mirem dins el directori del frontend trobarem els fitxers separats en una carpeta per guardar les imatges que es pugin des de la web (/public/upload/anuncis), la carpeta d’assets (on es guarden els diferents fitxers d’estils), a components es guarden els fitxers que es repeteixin al llarg de l’aplicació (com el header o footer), la carpeta de pages on hi ha tots els fitxers .html i per últim la carpeta de scripts amb tota la lògica del frontend (es guarden el .js

A continuació es mostra l’estructura seguida en aquest projecte.

None

```
servei-web/  
├─ backend/  
│   ├─ app.js  
│   ├─ config/  
│   │   └─ db.js  
│   ├─ models/  
│   ├─ routes/  
│   ├─ controllers/  
│   ├─ middlewares/  
│   ├─ docs.js  
│   └─ openapi.yaml  
└─ frontend/  
    ├─ public/upload/anuncis/  
    ├─ src/  
    │   ├─ assets/  
    │   │   └─ styles.css  
    │   ├─ components/  
    │   ├─ pages/  
    │   └─ scripts/  
    └─ index.html
```

Codi 6.1. Estructura del projecte. Font: creació pròpia.

Aquesta estructura redueix millora la reutilització i permet canviar o ampliar mòduls (p. ex. afegir filtres o nous endpoints) sense impactar la resta del sistema.

El codi està documentat allà on cal context: comentaris de bloc expliquen la lògica, supòsits i paràmetres; a CSS s'anoten les decisions del disseny adaptable. I al backend s'explica el comportament de filtres i estats en queries.

CSS

```
/* Mòbils */  
/* Quan la finestra < 600px, 1 sola columna */  
@media (max-width: 600px) {  
  .ads-grid {  
    grid-template-columns: 1fr;  
  }  
  .ads-grid .card-img img {  
    width: 100%;  
  }  
}
```

Codi 6.2. Exemple d'explicació del codi en els estils. Font: creació pròpia.

Disseny web responsiu

Com es pot comprovar a les següents captures de pantalla, la plataforma s'adapta completament a mides de pantalles petites i grans.

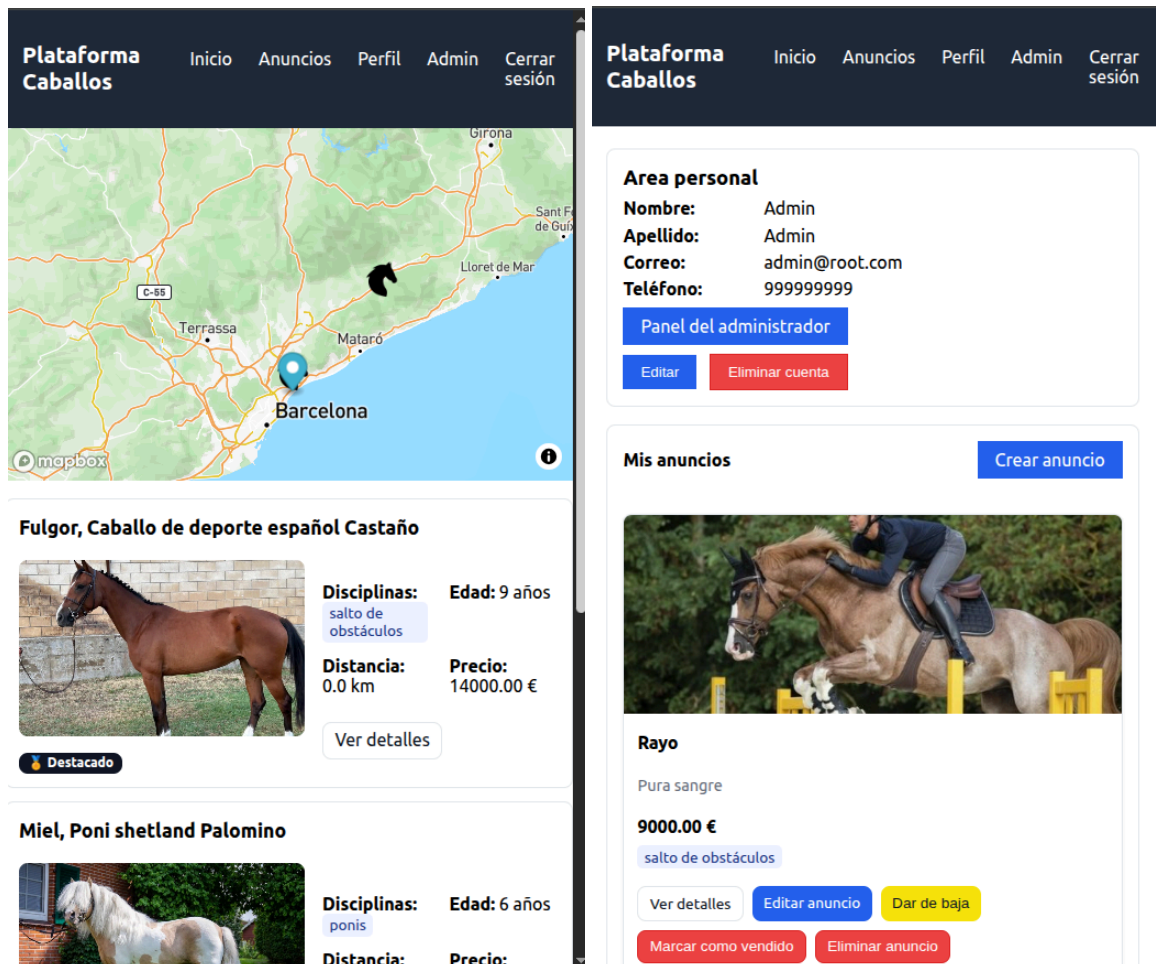


Figura 6.1. Captures de pantalla que demostren l'adaptabilitat de la plataforma Font: elaboració pròpia

7. SOSTENIBILITAT

En aquest apartat s'analitzen els principals aspectes relacionats amb la sostenibilitat i la responsabilitat del projecte. S'inclou l'avaluació de l'impacte ambiental, econòmic i social, una breu anàlisi d'igualtat de gènere, les implicacions ètiques i la relació amb els Objectius de Desenvolupament Sostenible (ODS).

7.1. MATRIU DE SOSTENIBILITAT

Impacte ambiental

Durant el desenvolupament del projecte, l'impacte ambiental principal prové de l'ús d'ordinadors i de la infraestructura necessària per al treball i les reunions virtuals. L'ús setmanal estimat de l'ordinador ha estat d'entre 20 i 25 hores, amb reunions periòdiques en línia de 1-2 hores mensuals. Això implica un consum mensual aproximat de 6 a 8 kWh, que equival a unes emissions de 2,5 a 3,5 kg de CO₂, segons els factors d'emissió mitjans de la generació elèctrica a Espanya.

En la futura fase de desplegament, la plataforma requerirà un servidor actiu les 24 hores durant tots els dies de la setmana. El consum mitjà anual d'aquest tipus d'equipament pot oscil·lar entre 500 i 700 kWh, amb emissions que se situarien al voltant de 200-300 kg de CO₂ anuals si no s'utilitzen fonts renovables. L'impacte pot reduir-se si s'opta per proveïdors de cloud hosting que garanteixin ús d'energies renovables.

Finalment, també cal considerar el consum energètic dels dispositius dels usuaris que accedeixin a la plataforma, així com la generació futura de residus electrònics associats a renovació d'infraestructura.

Impacte econòmic

En aquest projecte, el servidor s'ha implementat i executat directament a l'ordinador personal utilitzat pel desenvolupament. Això redueix de manera significativa el cost econòmic, ja que no s'han contractat serveis externs de hosting ni infraestructures al núvol:

- Consum energètic: l'ús del portàtil tant per al desenvolupament com per executar el servidor representa entre 6 i 8 kWh mensuals, amb un cost aproximat d'1,5 a 2 € al mes, depenent del preu de l'electricitat.
- Manteniment i evolució: no hi ha una despesa addicional significativa, més enllà de les actualitzacions periòdiques de programari i del sistema operatiu, que són gratuïtes.
- Equipament: per al desenvolupament del projecte, s'ha utilitzat un ordinador portàtil de gamma alta (1500 - 2000 €), tot i que seria suficient un ordinador de gamma mitja per a poder desenvolupar el projecte sense tenir cap problema.

Aquest model permet reduir despeses inicials i fer viable el projecte sense inversió externa. Tanmateix, en cas de desplegar la plataforma a llarg termini i amb un volum alt

d'usuaris, seria recomanable valorar la migració cap a un servei de cloud hosting per garantir una millor escalabilitat i disponibilitat.

Impacte social

El projecte té un impacte positiu en el sector eqüestre, ja que facilita la connexió entre venedors i compradors de cavalls, a més de promoure la visibilitat d'esdeveniments i activitats relacionades. Això contribueix a la professionalització i digitalització d'un sector tradicionalment menys digitalitzat.

També pot reduir costos de transacció i millorar la transparència, afavorint la confiança entre les parts. Tanmateix, cal tenir en compte la bretxa digital: alguns usuaris poden tenir dificultats d'accés o menys experiència en plataformes digitals, cosa que requerirà una atenció especial a la simplicitat i accessibilitat de la interfície.

7.2. ANÀLISI D'IGUALTAT DE GÈNERE

El desenvolupament del projecte ha tingut en compte l'ús d'un llenguatge inclusiu a la documentació i a la plataforma, utilitzant formes genèriques quan ha estat necessari.

Tot i que no hi ha funcionalitats específiques relacionades amb la perspectiva de gènere, la plataforma està dissenyada perquè qualsevol persona pugui utilitzar-la amb igualtat d'oportunitats, independentment del gènere. En futures versions, es podria valorar afegir opcions de personalització en els perfils d'usuari que reflecteixin preferències d'identitat o representació.

7.3. IMPLICACIONS ÈTIQUES

El projecte es regeix pels valors de transparència, seguretat i privadesa. En concret:

- Les dades dels usuaris (correu, contrasenyes, informació de contacte) s'emmagatzemen amb garanties de seguretat, utilitzant xifrat i bones pràctiques de gestió de credencials.
- Es garanteix la no discriminació en l'ús de la plataforma, mantenint un entorn inclusiu i accessible.
- Es promou un ús responsable de la tecnologia, amb criteris de sostenibilitat i eficiència energètica.

Aquestes accions estan alineades amb els principis ètics establerts per la UPC i amb bones pràctiques internacionals en desenvolupament de programari.

7.4. RELACIÓ AMB ELS OBJECTIUS DE DESENVOLUPAMENT SOSTENIBLE

El projecte es relaciona especialment amb els següents ODS (Objectius de Desenvolupament Sostenible):

- ODS 5 (Igualtat de gènere): es promou l'ús d'un llenguatge inclusiu i es garanteix que la plataforma sigui accessible per a qualsevol usuari sense discriminació.
- ODS 9 (Indústria, innovació i infraestructura): el projecte contribueix a digitalitzar el sector eqüestre, oferint una infraestructura tecnològica innovadora que millora l'eficiència i l'accés a la informació.
- ODS 12 (Consum i producció responsables): en promoure un mercat digital per a la venda de cavalls, es fomenta un consum més transparent i responsable, reduint la necessitat de desplaçaments físics i facilitant la comparació d'ofertes.

En conjunt, el projecte té un impacte ambiental moderat, un cost econòmic baix i beneficis socials clars en digitalització, accessibilitat i innovació dins el sector.

CONCLUSIONS

El projecte demostra la viabilitat tècnica d'una plataforma de compravenda de cavalls que compleix els requisits essencials: registre i gestió d'usuaris, cicle de vida dels anuncis (alta, edició, venut i baixa), validació editorial per part de l'administració, i cerca amb filtres avançats i mapa. L'arquitectura MVC, la separació clara de capes i l'ús de MariaDB per al model relacional garanteixen consistència, integritat i traçabilitat. La documentació d'API amb OpenAPI/Swagger i l'organització modular de rutes, controladors i models faciliten proves, mantenibilitat i evolució. El disseny responsiu i la interfície basada en components asseguruen una experiència d'usuari homogènia i coherent amb els objectius del domini.

Alhora, s'identifiquen àrees de millora que defineixen el marge d'evolució: la incorporació d'un mòdul d'esdeveniments eqüestres amb calendari i geolocalització, el sistema de favorits i alertes, i la gestió d'espais patrocinats/slots per a destacats. També es veu oportú consolidar proves automatitzades d'extrem a extrem, integrar mètriques d'ús per guiar decisions, i reforçar accessibilitat, internacionalització, seguretat i auditoria. En síntesi, el projecte assoleix l'estat funcional previst i estableix una base arquitectònica robusta; l'adopció progressiva d'aquestes línies de futur incrementarà el valor per a usuaris i administradors sense comprometre qualitat ni mantenibilitat.

BIBLIOGRAFIA

- [1] fabrizioscoglio. "Qué es la metodología Kanban y cómo funciona" <https://www.fabrizioscoglio.com/metodo-kanban/>
- [2] aws. "En qué consiste Scrum?" <https://aws.amazon.com/es/what-is/scrum/>
- [3] trello. <https://trello.com/es>
- [4] GanttPro. <https://app.ganttpro.com/>
- [5] draw.io. <https://www.drawio.com/>
- [6] surática. "¿Qué es el Frontend?" <https://www.suratica.es/que-es-el-frontend/>
- [7] Medium. "Demystifying MVC: Understanding the Model-View-Controller Architecture" <https://medium.com/@finzyphinzy/demystifying-mvc-understanding-the-model-view-controller-architecture-85c88a558951>
- [8] plantUML. <https://plantuml.com/>
- [9] balsamiq. <https://balsamiq.com/>
- [10] dbeaver. <https://dbeaver.io/download/>
- [11] nodejs. <https://nodejs.org/en/download>
- [12] mariadb. <https://mariadb.com/get-started-with-mariadb/#linux>
- [13] npm.documentation. <https://docs.npmjs.com/cli/v8/commands/npm>
- [14] express. <https://expressjs.com/>
- [15] npm.dotenv. <https://www.npmjs.com/package/dotenv>
- [16] stackoverflow. "How do I create a MySQL connection pool while working with NodeJS and Express?" <https://stackoverflow.com/questions/37102364/how-do-i-create-a-mysql-connection-pool-while-working-with-nodejs-and-express>
- [17] docs.npmjs. "Specifying dependencies and devDependencies in a package.json file" <https://docs.npmjs.com/specifying-dependencies-and-devdependencies-in-a-package-json-file>
- [18] github.documentation. <https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github>
- [19] swagger.io.specification <https://swagger.io/specification/>
- [20] GitHub. <https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository>
- [21] DigitalOcean. "How To Import and Export Databases in MySQL or MariaDB" <https://www.digitalocean.com/community/tutorials/how-to-import-and-export-databases-in->

Plataforma de Comerç Online per a la Venda de Cavalls
Francesco Oncins Spedo

[mysql-or-mariadb](#)